SLOT 3
DART ESSENTIALS

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

- Hiểu cấu trúc chuẩn **main.dart**, **clean code** và quy luật thực chi
- Functions & Functional Patterns
- Classes & Constructors
- Collections & Operations

- Hiểu được **code Flutter đang viết thực chất là gì**
- Không còn "viết theo mẫu" mà **biết phân tích – chỉnh sửa – mở rộng**
- Chuẩn bị tư duy cho **Widget, State, UI logic ở các slot sau**

A. HIỂU CẤU TRÚC CHUẨN MAIN.DART, CLEAN CODE VÀ QUY LUẬT THỰC CHI

1. TRÌNH TỰ HOẠT ĐỘNG CỦA FLUTTER

Luồng tổng quát

```
main()
↓
runApp()
↓
Root Widget
↓
build()
↓
Widget Tree
↓
Render UI
```

Quan trọng

- `build()` có thể chạy **nhiều lần**
- Logic trong `build()` = ❌ thảm họa

2. KIẾN TRÚC CLEAN CODE TỐI THIỂU (PHÙ HỢP SLOT 3)

```
lib/
├── main.dart          # Entry point
├── app.dart           # Root widget
├── screens/           # UI
├── services/          # Logic xử lý
├── models/            # Data
```

Quy tắc phụ thuộc

```
UI → Logic → Model
```

❌ Không được ngược lại

3. BÀI HỌC CỐT LÕI CHO SINH VIÊN

Nếu chỉ nhớ **5 điều về `main.dart`**, hãy nhớ:

1. `main.dart` chỉ để **khởi động app**
2. Không viết logic trong `main()`
3. Không viết logic trong `build()`
4. Logic phải là **Dart thuần**
5. Code càng tách → app càng sống lâu

4. NHỮNG QUY LUẬT BẤT BIẾN CỦA FLUTTER

1 UI có thể rebuild bất kỳ lúc nào

👉 Logic không được phụ thuộc vào `build()`

PhucPT10@fpt.edu.vn

2 Widget là immutable

👉 Trạng thái phải được quản lý riêng

3 Logic thuần Dart sống độc lập

👉 Test được – reuse được – maintain được

🧠 5. VAI TRÒ THỰC SỰ CỦA `main.dart` TRONG FLUTTER

`main.dart` KHÔNG phải nơi viết logic nghiệp vụ.

Nó là **điểm khởi động (entry point)** của toàn bộ ứng dụng Flutter.

Triết lý cốt lõi

- `main.dart` chịu trách nhiệm:
  - 🚀 Khởi chạy app
  - 🧭 Gắn root widget
  - ⚙️ Thiết lập cấu hình ban đầu
- **KHÔNG**:
  - Xử lý business logic
  - Viết thuật toán
  - Viết code xử lý dữ liệu phức tạp

👉 Nếu `main.dart` càng ngắn → **kiến trúc càng sạch**.

🧩 6. CẤU TRÚC TIÊU CHUẨN CỦA `main.dart`

Một `main.dart` **chuẩn Clean Code** thường có 4 phần:

```
import 'package:flutter/material.dart';
import 'app.dart';

void main() {
  runApp(const MyApp());
}
```

Hoặc chi tiết hơn:

```
void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const MyApp());
}
```

◈ PHẦN 1 – IMPORTS

```
import 'package:flutter/material.dart';
import 'app.dart';
```

Ý nghĩa

- Import **framework**
- Import **root widget**

Clean Code Rule

- ❌ Không import:
  - Service
  - Logic
  - Model
- ✅ Chỉ import những gì **cần để khởi chạy app**

👉 `main.dart` **không phụ thuộc business layer**

◈ PHẦN 2 – `main()` FUNCTION (ENTRY POINT)

```
void main() {
  runApp(const MyApp());
}
```


PhucPT10@fpt.edu.vn

Quy luật thực thi

- `main()` là **hàm đầu tiên chạy**
- Flutter runtime gọi `main()`
- Sau đó:
  - Khởi tạo engine
  - Gắn widget tree

Triết lý Clean Code

- `main()`:
  - Ngắn
  - Không logic
  - Không điều kiện phức tạp

❌ Sai lầm sinh viên hay mắc:

```
void main() {
  if (userLoggedIn) {
    runApp(HomeApp());
  } else {
    runApp(LoginApp());
  }
}
```

👉 Logic điều hướng **KHÔNG thuộc `main()`**

◈ PHẦN 3 – `runApp()` & ROOT WIDGET

```
runApp(const MyApp());
```

Ý nghĩa

- Gắn **root widget** vào Flutter engine
- Tạo **widget tree đầu tiên**

Quy luật Flutter

- Mọi thứ trong Flutter = Widget
- `MyApp` là **đỉnh cây (root)**

◈ PHẦN 4 – ROOT WIDGET (`MyApp`)

Ví dụ:

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: HomeScreen(),
    );
  }
}
```

Vai trò

- Thiết lập:
  - Theme
  - Route
  - App-level config

Clean Code Rule

- ❌ Không xử lý logic
- ✅ Chỉ:
  - Kết nối UI
  - Gắn màn hình đầu tiên

PhucPT10@fpt.edu.vn

📦 3. TRIẾT LÝ TÁCH LOGIC KHỎI UI (CORE CỦA CLEAN CODE)

❌ CÁCH SAI (RẤT PHỔ BIẾN)

```
onPressed: () {
  if (age > 18 && score > 50) {
    result = 'Pass';
  } else {
    result = 'Fail';
  }
}
```

👉 Logic **bị nhét trong UI**

✅ CÁCH ĐÚNG (TÁCH LOGIC)

Logic thuần Dart

```
bool isPassed(int age, int score) {
  return age > 18 && score > 50;
}
```

UI chỉ gọi

```
onPressed: () {
  final passed = isPassed(age, score);
}
```

Triết lý

UI	Logic
Hiển thị	Quyết định
Nhận input	Xử lý
Trigger	Tính toán

B. DART CORE

1 FUNCTIONS & FUNCTIONAL PATTERNS

(Hàm và cách tổ chức logic xử lý)

◈ Khái niệm cốt lõi cần nắm

- Hàm (function) là **khối code thực hiện một nhiệm vụ cụ thể**
- Hàm giúp:
  - Tránh lặp code
  - Dễ đọc, dễ bảo trì
- Các dạng hàm cơ bản trong Dart:
  - Hàm có / không có tham số
  - Hàm có / không có giá trị trả về
  - Arrow function (=>)
  - Anonymous function (hàm không tên)

🎯 Mục tiêu học tập

Sau phần này, sinh viên có thể:

- Tự viết hàm xử lý logic đơn giản (tính toán, kiểm tra điều kiện)
- Hiểu và sử dụng hàm trong `onPressed`, `onTap` của Flutter
- Đọc hiểu các đoạn code Flutter có nhiều hàm lồng nhau

🧠 Ý nghĩa

Hàm là **bước đầu tiên giúp sinh viên tư duy có cấu trúc**, tách UI và logic – một kỹ năng **bắt buộc khi làm Flutter thực tế**.

2 CLASSES & CONSTRUCTORS

(Lập trình hướng đối tượng cơ bản với Dart)

PhucPT10@fpt.edu.vn

Khái niệm cốt lõi cần nắm

- Class là **khuôn mẫu mô tả một đối tượng**
- Một class gồm:
  - Thuộc tính (variables)
  - Phương thức (functions)
- Constructor:
  - Dùng để **khởi tạo object**
  - Hiểu cách truyền dữ liệu khi tạo object
- Từ khóa `this`, `final`, `const` (ở mức cơ bản)

Mục tiêu học tập

Sau phần này, sinh viên có thể:

- Tạo class đơn giản (User, Product, Todo…)
- Hiểu vì sao Flutter widget là một class
- Biết cách truyền dữ liệu vào widget thông qua constructor

Ý nghĩa

Class & constructor giúp sinh viên **bước vào tư duy Flutter thực thụ**, vì **mọi widget đều là class**, mọi UI đều bắt đầu từ constructor.

3 COLLECTIONS & OPERATIONS

(List, Map, Set và xử lý dữ liệu)

Khái niệm cốt lõi cần nắm

- Collection là **tập hợp nhiều dữ liệu**
- Các loại phổ biến:
  - `List`: danh sách có thứ tự
  - `Map`: dữ liệu dạng key – value
- Các thao tác quan trọng:
  - Thêm, xóa, duyệt phần tử
  - `for`, `forEach`
  - `map()`, `where()` (ở mức nhập môn)

Mục tiêu học tập

Sau phần này, sinh viên có thể:

- Lưu và xử lý danh sách dữ liệu
- Hiểu cách Flutter hiển thị danh sách (`ListView.builder`)
- Biết biến dữ liệu thành UI (map data → widget)

Ý nghĩa

Đây là **cầu nối trực tiếp giữa Dart và UI Flutter**, giúp sinh viên hiểu:
"Vì sao có danh sách dữ liệu thì mới vẽ được danh sách giao diện".

MỐI LIÊN HỆ VỚI FLUTTER CƠ BẢN

Slot 3 giúp sinh viên:

- Hiểu logic bên trong `build()`
- Không còn sợ code Dart trong Widget
- Chuẩn bị cho:
  - State management cơ bản
  - Form, List, API, Model ở các slot sau

NGUỒN TÀI LIỆU THAM KHẢO UY TÍN (DỄ TIẾP CẬN)

1. **Dart Official Documentation (Beginner Friendly)**
   👉 https://dart.dev/guides
   (Chuẩn chính thức, có ví dụ rõ ràng)

PhucPT10@fpt.edu.vn

2. **Flutter Documentation – Dart Language Tour**
   👉 https://docs.flutter.dev/resources/learn-dart
   (Liên hệ trực tiếp giữa Dart và Flutter)
3. **Sách: Flutter Apprentice (Ray Wenderlich / Kodeco)**
   (Giải thích rất chậm, phù hợp sinh viên mới bắt đầu)
4. **YouTube: Flutter & Dart – Code With Andrea**
   (Giải thích tư duy, không chỉ cú pháp)
5. **YouTube: Academind – Dart & Flutter Basics**
   (Rõ ràng, nhiều ví dụ thực tế)

PhucPT10@fpt.edu.vn

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

1. FUNCTIONS & FUNCTIONAL PATTERNS

(Hàm & tư duy tổ chức logic)

A. KHÁI NIỆM QUAN TRỌNG (CORE CONCEPTS) – BẮT BUỘC PHẢI HIỂU

- **Function là đơn vị xử lý logic**, không phải là nơi viết UI
  → Tách rõ: *UI hiển thị* vs *logic xử lý*
- **Input (tham số) → Process → Output (giá trị trả về)**
  → Hiểu hàm như một "hộp xử lý"
- **Function ≠ chạy ngay**, chỉ chạy khi được gọi
  → Tránh hiểu nhầm "viết là chạy"
- **Callback**: function được truyền như dữ liệu
  → Xuất hiện rất nhiều trong Flutter (`onPressed`, `onTap`)
- **Anonymous function (hàm không tên)**
  → Hiểu để đọc code Flutter, **chưa cần tự viết phức tạp**

✅ **BẮT BUỘC HIỂU**

❌ **Chưa cần học sâu**: higher-order function nâng cao, functional programming phức tạp

B. KỸ NĂNG / THAO TÁC PHẢI LÀM ĐƯỢC

- Viết được function:
  - Có / không có tham số
  - Có / không có giá trị trả về
- Gọi function đúng ngữ cảnh (đặc biệt trong Flutter)
- Truyền function làm callback cho widget:
  - `onPressed: () { ... }`
- Biết **khi nào nên viết function riêng** thay vì nhồi logic trong `build()`

C. LỖI THƯỜNG GẶP CỦA SINH VIÊN MỚI

- ❌ Viết toàn bộ logic trong `build()`
  → *Sai tư duy*: build chỉ để dựng UI
- ❌ Không hiểu tham số → truyền đại cho chạy
  → *Hiểu cú pháp nhưng không hiểu dữ liệu*
- ❌ Gọi function ngay khi truyền callback
  (`onPressed: myFunc()` thay vì `myFunc`)
  → *Không phân biệt gọi hàm vs truyền hàm*

D. BEST PRACTICES (THỰC HÀNH CHUẨN)

- Mỗi function **chỉ làm một việc rõ ràng**
- Đặt tên function **diễn tả hành động**
- Tách logic xử lý ra khỏi widget UI càng sớm càng tốt

Ý NGHĨA

Nếu không nắm function & callback, sinh viên sẽ **không hiểu onPressed, không kiểm soát được logic UI**, và học Flutter chỉ ở mức "copy – paste".

2. CLASSES & CONSTRUCTORS

(Tư duy đối tượng – nền tảng của Flutter)

A. KHÁI NIỆM QUAN TRỌNG (CORE CONCEPTS) – BẮT BUỘC PHẢI HIỂU

- **Class là bản thiết kế**, object là sản phẩm được tạo ra từ class
- **Thuộc tính mô tả dữ liệu**, method mô tả hành vi
- **Constructor dùng để khởi tạo dữ liệu ban đầu**

PhucPT10@fpt.edu.vn

- **Widget Flutter = class Dart**
- **Truyền dữ liệu qua constructor** là cách Flutter hoạt động

✅ **BẮT BUỘC HIỂU**

❌ **Chưa cần học sâu**: inheritance phức tạp, abstract, mixin

◈ B. KỸ NĂNG / THAO TÁC PHẢI LÀM ĐƯỢC

- Tạo class đơn giản (model dữ liệu)
- Viết constructor:
  - Nhận tham số
  - Gán cho thuộc tính
- Tạo object từ class
- Truyền dữ liệu vào widget qua constructor

◈ C. LỖI THƯỜNG GẶP CỦA SINH VIÊN MỚI

- ❌ Không hiểu constructor, chỉ copy theo mẫu
  → *Mất gốc tư duy đối tượng*
- ❌ Để mọi thứ là `var`, không kiểm soát dữ liệu
  → *Thiếu ý thức về dữ liệu đầu vào*
- ❌ Không hiểu vì sao widget cần constructor
  → *Không hiểu data flow Flutter*

◈ D. BEST PRACTICES (THỰC HÀNH CHUẨN)

- Dùng `final` cho dữ liệu không thay đổi
- Constructor rõ ràng, ít tham số
- Class dùng để **đóng gói dữ liệu**, không nhồi logic UI

🎓 Ý NGHĨA

Không hiểu class & constructor = **không hiểu Flutter**, vì mọi widget, state, dữ liệu đều xoay quanh object & constructor.

II. COLLECTIONS & OPERATIONS

(Xử lý dữ liệu để hiển thị UI)

◈ A. KHÁI NIỆM QUAN TRỌNG (CORE CONCEPTS) – BẮT BUỘC PHẢI HIỂU

- **List = danh sách có thứ tự**
- **Map = key – value (dữ liệu có nhãn)**
- **UI Flutter thường được render từ Collection**
- **Dữ liệu → duyệt → biến thành widget**

✅ **BẮT BUỘC HIỂU**

❌ **Chưa cần học sâu**: collection nâng cao, immutable collection, performance tuning

◈ B. KỸ NĂNG / THAO TÁC PHẢI LÀM ĐƯỢC

- Khai báo và sử dụng List / Map
- Thêm, xóa, truy cập phần tử
- Duyệt List bằng `for`, `map()`
- Dùng dữ liệu từ List để render UI (`ListView.builder`)

◈ C. LỖI THƯỜNG GẶP CỦA SINH VIÊN MỚI

- ❌ Không hiểu List chứa object gì
  → *Chỉ biết "có List" nhưng không hiểu dữ liệu*

PhucPT10@fpt.edu.vn

- ❌ Lạm dụng `map()` nhưng không hiểu nó trả về gì
  → *Học vẹt cú pháp*
- ❌ Nhầm Map với List
  → *Sai tư duy cấu trúc dữ liệu*

🔷 D. BEST PRACTICES (THỰC HÀNH CHUẨN)

- Luôn xác định rõ:
  - List chứa kiểu dữ liệu gì
- Tách dữ liệu và UI
- Viết code xử lý List **trước khi render UI**

🎓 Ý NGHĨA

Nếu không hiểu Collection, sinh viên sẽ **không render được danh sách**, không hiểu ListView, và "kẹt cứng" khi làm app thực tế.

✅ TỔNG KẾT MỤC TIÊU SƯ PHẠM SLOT 3

Sau Slot 3, sinh viên **BẮT BUỘC PHẢI**:

- Hiểu và viết được function đúng vai trò
- Hiểu class & constructor để đọc – viết widget
- Xử lý List / Map để render UI Flutter

❌ **Chưa cần**:

- OOP nâng cao
- Functional programming chuyên sâu
- State management phức tạp

PhucPT10@fpt.edu.vn

III. ỨNG DỤNG THỰC TIỄN

1 ỨNG DỤNG SLOT 3 TRONG PHÁT TRIỂN DỰ ÁN FLUTTER

1. Functions & Functional Patterns – DÙNG Ở ĐÂU?

Trong Flutter thực tế, function xuất hiện ở:

- `build()` → gọi các function chuẩn bị dữ liệu
- `onPressed`, `onTap` → callback xử lý hành động
- Logic kiểm tra dữ liệu trước khi hiển thị UI

Vai trò thực tế

- **Tách logic ra khỏi UI**
- Giúp widget:
  - Gọn
  - Dễ đọc
  - Dễ sửa

Nếu **không nắm function**:

- Sinh viên sẽ:
  - Nhét toàn bộ logic vào `build()`
  - Không kiểm soát được callback
  - Sợ code dài

2. Classes & Constructors – DÙNG Ở ĐÂU?

Trong Flutter thực tế:

- Mỗi **widget là một class**
- Mỗi **dữ liệu hiển thị nên là một class (model)**

Vai trò thực tế

- Giúp:
  - Dữ liệu có cấu trúc
  - Truyền dữ liệu rõ ràng qua constructor
- Tránh:
  - Dữ liệu dạng `Map<String, dynamic>` lung tung
  - Không biết key nào tồn tại

Nếu **không nắm class & constructor**:

- Sinh viên:
  - Không hiểu widget nhận dữ liệu từ đâu
  - Không hiểu data flow
  - Copy code mà không biết sửa

3. Collections & Operations – DÙNG Ở ĐÂU?

Trong Flutter thực tế:

- List → hiển thị danh sách (`ListView`)
- Map → dữ liệu dạng cấu trúc
- `map()`, `where()` → biến dữ liệu thành widget

Vai trò thực tế

- **Chuẩn bị dữ liệu trước khi build UI**
- Giúp UI:
  - Chỉ tập trung hiển thị
  - Không xử lý logic phức tạp

Nếu **không nắm collection**:

- Sinh viên:
  - Không render được danh sách

PhucPT10@fpt.edu.vn

    - Lẫn lộn dữ liệu – UI
    - Sợ ListView.builder

2 CÁC TÌNH HUỐNG THỰC TẾ (REAL-WORLD SCENARIOS)

TÌNH HUỐNG 1: APP QUẢN LÝ CÔNG VIỆC CÁ NHÂN (TODO APP)

Bối cảnh

- App sinh viên
- Danh sách công việc
- Có nút hoàn thành / xóa

Vấn đề ban đầu

- Logic xử lý click viết trực tiếp trong `build()`
- Code lặp lại nhiều lần
- Khó sửa khi thêm chức năng mới

Áp dụng Slot 3

- **Function**:
    - Tách hàm xử lý hoàn thành công việc
- **Class**:
    - `Todo` làm model dữ liệu
- **List**:
    - Lưu danh sách công việc
    - `map()` để render UI

Mục tiêu đạt được

- Code rõ vai trò:
    - UI chỉ hiển thị
    - Logic nằm trong function
- Sinh viên:
    - Hiểu luồng xử lý
    - Dễ mở rộng thêm tính năng

TÌNH HUỐNG 2: APP HIỂN THỊ DANH SÁCH SẢN PHẨM (MVP NHỎ)

Bối cảnh

- MVP cho startup nhỏ
- Hiển thị danh sách sản phẩm
- Có lọc theo giá

Vấn đề ban đầu

- Dữ liệu để dạng Map rời rạc
- Lọc dữ liệu ngay trong `build()`
- UI bị rối

Áp dụng Slot 3

- **Class**:
    - `Product` làm model
- **Function**:
    - Hàm lọc danh sách sản phẩm
- **Collection**:
    - `where()` để lọc dữ liệu trước khi render

Mục tiêu đạt được

- Dữ liệu rõ ràng
- UI đơn giản
- Sinh viên hiểu:
  "UI chỉ là kết quả của dữ liệu đã xử lý"

PhucPT10@fpt.edu.vn

🧩 TÌNH HUỐNG 3: APP NỘI BỘ – DANH SÁCH NHÂN VIÊN

🔷 Bối cảnh

- App nội bộ công ty
- Hiển thị danh sách nhân viên
- Nhấn vào xem chi tiết

❗ Vấn đề ban đầu

- Không hiểu callback
- Không biết truyền dữ liệu qua widget

🛠 Áp dụng Slot 3

- **Function callback**:
  - Xử lý khi click item
- **Constructor**:
  - Truyền `Employee` vào widget chi tiết
- **List**:
  - Render danh sách bằng `ListView.builder`

✅ Mục tiêu đạt được

- Sinh viên hiểu callback
- Hiểu:
  - Dữ liệu đi từ đâu → đến đâu
- Không còn sợ widget nhận tham số

3 CASE STUDY – ÁP DỤNG SLOT 3 TRONG DỰ ÁN

🔑 Bối cảnh

- App quản lý chi tiêu cá nhân cho sinh viên

❗ Vấn đề ban đầu

- Logic tính tổng tiền viết trong `build()`
- Dữ liệu chi tiêu để dạng Map
- UI khó đọc, khó sửa

🛠 Áp dụng Slot 3

- **Class**:
  - `Expense` làm model
- **Function**:
  - Tính tổng chi tiêu
- **Collection**:
  - List lưu các khoản chi
  - `map()` để render UI

✅ Kết quả đạt được

- Code:
  - Gọn
  - Có cấu trúc
- Sinh viên:
  - Hiểu rõ luồng xử lý
  - Không còn "code mù"

4 BÀI HỌC RÚT RA (LESSONS LEARNED)

🔍 Điều quan trọng nhất

Dart core không phải lý thuyết – nó là logic sống của Flutter

❌ Sai lầm phổ biến của sinh viên

- Viết logic trong `build()`

PhucPT10@fpt.edu.vn

- Không dùng class cho dữ liệu
- Dùng List mà không hiểu nó chứa gì
- Học cú pháp `map()` nhưng không hiểu kết quả trả về

✅ Nguyên tắc cần duy trì lâu dài

- **Tách logic – UI**
- **Dữ liệu phải có cấu trúc (class)**
- **Ưu tiên code dễ đọc hơn code ngắn**
- Chuẩn bị dữ liệu **trước khi build UI**

PhucPT10@fpt.edu.vn

IV. MỘT SỐ NỘI DUNG CHI TIẾT

◈ PHẦN A – FUNCTIONS & FUNCTIONAL PATTERNS

I. BẢN CHẤT CỦA FUNCTION TRONG DART

◈ Function là gì?

- **Function là một khối logic thực hiện một nhiệm vụ cụ thể**
- Function giúp:
  - Gom logic lại
  - Đặt tên cho logic
  - Tái sử dụng logic

👉 Hiểu đơn giản:
Function = "Hành động được đặt tên"

◈ Vì sao Flutter dùng function ở khắp nơi?

Trong Flutter:

- `build()` là function
- `onPressed`, `onTap` là function
- `builder` trong ListView là function

👉 Flutter **không gọi bạn vẽ UI**, Flutter:
"Gọi function của bạn để lấy UI"

◈ Phân biệt 2 vai trò function (rất quan trọng)

- **Function xử lý logic**
  - Tính toán
  - Kiểm tra dữ liệu
- **Function phản hồi sự kiện (callback)**
  - Chạy khi user tương tác

❗ Nếu không phân biệt được → code rối, debug khó

🧠 Ghi chú sư phạm

Học function để **điều khiển hành vi của app**, không chỉ để "viết cho có".

II. CÁC LOẠI FUNCTION CỐT LÕI

1 Function có / không có tham số

Cú pháp:

```
void sayHello() {
  print("Hello");
}

void greet(String name) {
  print("Hello $name");
}
```

Bản chất:

- Tham số = dữ liệu đầu vào
- Giúp function linh hoạt

Liên hệ Flutter:

- Widget nhận dữ liệu qua constructor
- Function nhận dữ liệu qua tham số

❌ Sai lầm:

- Viết function nhưng hard-code dữ liệu bên trong

PhucPT10@fpt.edu.vn

2 Function có / không có giá trị trả về

Cú pháp:

```
int sum(int a, int b) {
  return a + b;
}
```

Bản chất:

- Function có thể **trả kết quả cho nơi gọi**
- Rất quan trọng khi:
  - Chuẩn bị dữ liệu trước khi build UI

Liên hệ Flutter:

- Hàm chuẩn bị list để hiển thị
- Hàm tính tổng, lọc dữ liệu

❌ Sai lầm:

- Dùng print thay vì return → không tái sử dụng được

3 Arrow Function (=>)

Cú pháp:

```
int doubleValue(int x) => x * 2;
```

Bản chất:

- Viết gọn cho function **chỉ có 1 biểu thức**
- Không phải "cú pháp mới", chỉ là **viết gọn**

Liên hệ Flutter:

- Hay gặp trong:
  - `map()`
  - callback đơn giản

❌ Sai lầm:

- Dùng arrow function cho logic dài → khó đọc

4 Anonymous Function (Hàm không tên)

Cú pháp:

```
() {
  print("Clicked");
}
```

Bản chất:

- Function dùng **ngay tại chỗ**
- Không cần tái sử dụng

Liên hệ Flutter:

```
onPressed: () {
  print("Button pressed");
}
```

❌ Sai lầm:

- Nhét quá nhiều logic vào anonymous function

5 Callback Function (CỰC KỲ QUAN TRỌNG)

Khái niệm:

- Callback = function **được truyền như dữ liệu**
- Flutter sẽ gọi function đó **khi có sự kiện**

Ví dụ:

```
void handleClick() {
  print("Clicked");
}

onPressed: handleClick
```


PhucPT10@fpt.edu.vn

❗ **Không phải** `handleClick()`
Sai lầm kinh điển:

- Gọi function ngay khi build → app chạy sai

🧠 Ghi chú sư phạm
Không hiểu callback = không hiểu Flutter
Slot 3 là nơi **chữa gốc lỗi này**

III. SAI LẦM PHỔ BIẾN (FUNCTION)

- Viết logic dồn trong `build()`
- Không tách function theo nhiệm vụ
- Nhầm:
  - Gọi function
  - Truyền function

◈ PHẦN B – CLASSES & CONSTRUCTORS

I. VÌ SAO CẦN CLASS TRONG FLUTTER?

◈ Class KHÔNG chỉ để "đúng OOP"

Class trong Flutter dùng để:

- Mô tả **dữ liệu có cấu trúc**
- Truyền dữ liệu rõ ràng
- Giảm lỗi runtime

👉 **Widget là class**
👉 **Dữ liệu nên là class**

🧠 Ghi chú sư phạm
Không dùng class cho dữ liệu = tự làm khó mình khi app lớn lên

II. CONSTRUCTORS

1 Default Constructor

```
class User {
  String name;
  User(this.name);
}
```

Bản chất:

- Constructor = cửa vào của object
- Bắt buộc dữ liệu phải đầy đủ khi tạo object

2 Named Constructor

```
class User {
  String name;
  User.guest() : name = "Guest";
}
```

Bản chất:

- Tạo object theo **ngữ cảnh khác nhau**

❌ Chưa cần lạm dụng – chỉ cần hiểu

3 const Constructor (giới thiệu nhẹ)

- Dùng khi object **không thay đổi**
- Giúp tối ưu Flutter widget

👉 Hiểu khái niệm, **chưa cần dùng nhiều**

PhucPT10@fpt.edu.vn

🧠 Ghi chú sư phạm

Constructor giúp **kiểm soát dữ liệu đầu vào**, tránh object "nửa vời".

III. OBJECT & VÒNG ĐỜI CƠ BẢN

- **Class**: bản thiết kế
- **Object**: sản phẩm cụ thể

```
User user = User("Phuc");
```

- Object được tạo khi:
  - Gọi constructor
- Object tồn tại để:
  - Cung cấp dữ liệu cho UI

IV. LỖI THƯỜNG GẶP (CLASS)

- Nhầm class và object
- Truyền dữ liệu bằng Map thay vì model
- Class phình to:
  - Vừa dữ liệu
  - Vừa UI
  - Vừa logic

◈ PHẦN C – COLLECTIONS & OPERATIONS

I. COLLECTION LÀ GÌ & VÌ SAO QUAN TRỌNG?

- Collection = tập hợp dữ liệu
- Flutter UI:
  Gần như luôn render từ collection

👉 Danh sách, grid, menu, list… đều từ List

II. LIST / MAP / SET

◈ List

- Dữ liệu có thứ tự
- Dùng cho:
  - Danh sách hiển thị

```
List<int> numbers = [1, 2, 3];
```

◈ Map

- Dữ liệu dạng key – value
- Dùng khi:
  - Truy cập theo tên

```
Map<String, String> user = {
  "name": "Phuc",
};
```

◈ Set

- Không trùng lặp
- Ít dùng ở Flutter cơ bản
- Biết khái niệm là đủ

III. DUYỆT & BIẾN ĐỔI DỮ LIỆU

1 for / for-in

```
for (var item in items) {
  print(item);
```


PhucPT10@fpt.edu.vn

```
}
```

2 map()

```
items.map((item) => Text(item.name)).toList();
```

Bản chất:

- Biến **dữ liệu → dữ liệu mới**
- Trong Flutter:
  - Data → Widget

❗ map() **không trả về widget**, mà trả về **Iterable**

3 where()

```
items.where((item) => item.price > 100);
```

- Lọc dữ liệu trước khi hiển thị

🧠 Ghi chú sư phạm

Chuẩn bị dữ liệu **TRƯỚC** build UI
build() chỉ để hiển thị

IV. LỖI PHỔ BIẾN (COLLECTION)

- Xử lý logic trong UI
- Không hiểu map() trả về gì
- Dùng sai loại collection

PhucPT10@fpt.edu.vn

V. KINH NGHIỆM THỰC TIỄN, TIPS

FUNCTIONS & FUNCTIONAL PATTERNS

KỸ THUẬT: TÁCH LOGIC BẰNG FUNCTION

KINH NGHIỆM THỰC TẾ TRONG DỰ ÁN FLUTTER

- Trong app thật, **function không dùng để viết cho đẹp**, mà để:
  - Tách logic ra khỏi widget
  - Giảm độ phức tạp của `build()`
- Thường dùng ở:
  - Chuẩn bị dữ liệu trước UI
  - Xử lý event (`onPressed`, `onTap`)
  - Kiểm tra điều kiện (validate, filter, tính toán)

Tình huống thực tế

- App sinh viên quản lý chi tiêu
- Ban đầu: tính tổng tiền ngay trong `build()`

Vấn đề

- `build()` dài
- Mỗi lần UI rebuild → logic chạy lại
- Khó debug khi số liệu sai

Vì sao dùng function?

- Logic tính toán **không thuộc UI**
- Function giúp test & sửa độc lập

LỖI THƯỜNG GẶP

Lỗi 1: Nhét toàn bộ logic vào build()

Nguyên nhân:

- Hiểu sai vai trò của `build()`
- Nghĩ "viết ở đâu cũng được miễn chạy"

Ví dụ:

```
Text(totalExpenses().toString());
```

→ nhưng `totalExpenses()` chứa cả logic lọc + tính toán

Cách khắc phục:

- Tách function xử lý dữ liệu **ngoài widget**

Phòng tránh:

- Luôn tự hỏi: *"Logic này có phải là UI không?"*

Lỗi 2: Function làm quá nhiều việc

Nguyên nhân:

- Không chia nhỏ trách nhiệm
- Sợ viết nhiều function

Cách khắc phục:

- 1 function = 1 nhiệm vụ

MẸO LẬP TRÌNH

- Đặt tên function theo **hành động**
- Đọc tên function phải đoán được nó làm gì
- Viết logic bằng function trước → sau đó mới gắn vào UI

BEST PRACTICES

- `build()` chỉ nên:

PhucPT10@fpt.edu.vn

    - Gọi function
    - Hiển thị kết quả
- Nếu function > 10–15 dòng → xem lại trách nhiệm

🎓 BÀI HỌC RÚT RA

Nếu build() khó đọc → Dart core đang có vấn đề
Function tốt = Flutter dễ thở

🔷 KỸ THUẬT: CALLBACK FUNCTION

✅ KINH NGHIỆM THỰC TẾ

- Callback là **xương sống của tương tác người dùng**
- Dùng khi:
    - User click
    - Widget con báo ngược kết quả lên widget cha

📍 **Bối cảnh**

- App danh sách công việc
- Mỗi item có nút "Hoàn thành"

❗ **Vấn đề**

- Sinh viên gọi function ngay khi build

🤔 **Vì sao callback được chọn?**

- Flutter **chỉ gọi callback khi sự kiện xảy ra**

⚠️ LỖI THƯỜNG GẶP

❌ **Lỗi kinh điển: onPressed: myFunc()**

🔍 Nguyên nhân:

- Không phân biệt:
    - Gọi function
    - Truyền function

🧪 Ví dụ:

```
onPressed: completeTask()
```

🛠️ Cách khắc phục:

```
onPressed: completeTask
```

🚫 Phòng tránh:

- Nhớ: **callback KHÔNG có dấu ()**

💡 MẸO

- Nếu callback cần tham số → dùng anonymous function

```
onPressed: () => completeTask(id)
```

🛡️ BEST PRACTICES

- Callback chỉ nên gọi function khác
- Không nhét logic dài trong anonymous function

🎓 BÀI HỌC RÚT RA

Không hiểu callback = không hiểu Flutter
Slot 3 là nơi sửa lỗi này tận gốc

🧱 CLASSES & CONSTRUCTORS

PhucPT10@fpt.edu.vn

🔷 KỸ THUẬT: CLASS LÀM MODEL DỮ LIỆU

✅ KINH NGHIỆM THỰC TẾ

- Trong dự án thật:
  - **Dữ liệu luôn có cấu trúc**
- Class giúp:
  - Code dễ đọc
  - Ít bug runtime
  - Truyền dữ liệu rõ ràng

📍 **Bối cảnh**

- App hiển thị danh sách sản phẩm

❗ **Vấn đề**

- Dữ liệu để dạng Map:

```
product["price"]
```

→ gõ sai key là crash

🤔 **Vì sao dùng class?**

- IDE hỗ trợ
- Biết rõ dữ liệu có gì

⚠️ LỖI THƯỜNG GẶP

❌ **Không dùng class cho dữ liệu**

🔍 Nguyên nhân:

- Ngại viết class
- Nghĩ Map là đủ

🛠 Khắc phục:

- Tạo model class ngay từ đầu

🚫 Phòng tránh:

- Danh sách dữ liệu > 2 field → dùng class

❌ **Class ôm quá nhiều trách nhiệm**

🔍 Nguyên nhân:

- Gom cả logic + UI + dữ liệu

🛠 Khắc phục:

- Class model **chỉ giữ dữ liệu**

💡 MẸO

- Mỗi model → 1 file Dart
- Đặt tên class là danh từ (User, Product)

🛡 BEST PRACTICES

- Dùng `final` cho field không thay đổi
- Constructor rõ ràng, ít tham số

🎓 BÀI HỌC RÚT RA

Dữ liệu không có cấu trúc → Flutter càng viết càng mệt

🔷 KỸ THUẬT: CONSTRUCTOR & TRUYỀN DỮ LIỆU

✅ KINH NGHIỆM THỰC TẾ

- Truyền dữ liệu qua constructor là:
  - Cách Flutter hoạt động
  - Cách UI nhận dữ liệu

PhucPT10@fpt.edu.vn

📍 **Bối cảnh**

- Nhấn item → mở màn hình chi tiết

❗ **Vấn đề**

- Truyền từng field lẻ tẻ

😕 **Giải pháp**

- Truyền cả object qua constructor

⚠ LỖI THƯỜNG GẶP

- Truyền quá nhiều tham số
- Không hiểu dữ liệu đến từ đâu

🛡 BEST PRACTICES

- Truyền object thay vì nhiều biến rời
- Constructor = "cổng dữ liệu" của widget

🎓 BÀI HỌC RÚT RA

Constructor giúp kiểm soát data flow – đừng xem nhẹ

📚 COLLECTIONS & OPERATIONS

🔷 KỸ THUẬT: LIST & MAP TRONG RENDER UI

✅ KINH NGHIỆM THỰC TẾ

- UI Flutter **luôn render từ List**
- Dữ liệu phải được xử lý **trước khi build**

📍 **Bối cảnh**

- ListView hiển thị dữ liệu lọc

❗ **Vấn đề**

- Lọc dữ liệu ngay trong `build()`

😕 **Giải pháp**

- Dùng function + `where()` trước UI

⚠ LỖI THƯỜNG GẶP

❌ **Không hiểu map() trả về gì**

🔍 Nguyên nhân:

- Học cú pháp, không hiểu bản chất

🧪 map() → **Iterable**, không phải Widget

🛠 Khắc phục:

- Nhớ: `map()` + `toList()`

❌ **Dùng sai collection**

- Dùng Map thay vì List
- Không hiểu dữ liệu có thứ tự hay không

💡 MẸO

- Trước khi code UI:
  - Viết code xử lý List riêng
- Print dữ liệu trung gian để kiểm tra

🛡 BEST PRACTICES

- UI không xử lý logic phức tạp

PhucPT10@fpt.edu.vn

- Collection dùng đúng mục đích:
  - List → danh sách
  - Map → tra cứu theo key

🎓 BÀI HỌC RÚT RA

UI chỉ là kết quả – Dart core mới là nguyên nhân

🎯 TỔNG KẾT PROMPT 5 – THÔNG ĐIỆP CHỐT

- Dart core yếu → Flutter rất khó
- Dart core tốt → Flutter nhẹ, logic rõ

👉 Nếu chỉ nhớ **3 nguyên tắc sống còn**:

1. **Tách logic khỏi UI**
2. **Dữ liệu phải có cấu trúc**
3. **Ưu tiên code dễ đọc hơn code "viết nhanh"**

PhucPT10@fpt.edu.vn

EXERCISE

📒 DỰ ÁN THỰC HÀNH 1

1 TỔNG QUAN DỰ ÁN

BÀI TẬP 2 – HIỂU ĐÚNG FUNCTION TRONG DART (LOGIC CORE)

🎯 MỤC TIÊU BÀI TẬP

Sau bài này, sinh viên **PHẢI hiểu rõ**:

- ✅ Function **chỉ xử lý logic**, không phải nơi viết UI
- ✅ Function hoạt động theo mô hình:
  Input → Process → Output
- ✅ Function **không tự chạy**, chỉ chạy khi được gọi
- ✅ Callback = function được truyền như dữ liệu
- ✅ Anonymous function dùng để **đọc hiểu code Flutter** (chưa cần viết phức tạp)

⚠️ Nếu sinh viên chưa hiểu bài này → sẽ **rất khó học Widget, onPressed, onTap ở Slot sau**

🧠 TƯ DUY TRỌNG TÂM

❝ Function giống như một chiếc máy xử lý
Đưa dữ liệu vào → xử lý → trả kết quả ❞

Không phải:

- Viết cho chạy ngay
- Viết cho in ra màn hình
- Viết lẫn UI & logic

1 TỔNG QUAN BÀI TẬP

📌 Tên bài tập

Function Processor – Xử lý dữ liệu bằng Dart Function

📖 Mô tả

Viết các function Dart để xử lý dữ liệu số và chuỗi,
sau đó **gọi function từ `main()`** để kiểm tra kết quả bằng `print()`.

👉 **Không có UI**
👉 **Không có Widget**
👉 **Chỉ tập trung vào logic**

2 KIẾN THỨC ĐƯỢC CỦNG CỐ

Nội dung	Mục đích
Function thường	Hiểu input → output
Return value	Không lạm dụng print
Gọi function	Hiểu "viết ≠ chạy"
Callback	Chuẩn bị cho Flutter
Anonymous function	Đọc hiểu onPressed

3 CHUẨN BỊ MÔI TRƯỜNG (VS CODE – RÕ TỪNG BƯỚC)

🔷 Bước 1: Mở project Flutter đã tạo

- Mở **VS Code**
- Chọn **File → Open Folder**
- Mở thư mục project Flutter (vd: `flutter_application_3`)

PhucPT10@fpt.edu.vn

🔷 Bước 2: Mở file `lib/main.dart`

📁 Cây thư mục:

```
lib/
 └── main.dart
```

👉 **Toàn bộ bài tập 2 làm trong `main.dart`**

4 PHẦN 1 – FUNCTION LÀ "HỘP XỬ LÝ" (INPUT → OUTPUT)

📌 Mục tiêu

Hiểu function:

- Nhận dữ liệu
- Xử lý
- Trả kết quả

🛠 Thao tác

Bước 1: Viết function xử lý logic

```
int calculateTotal(int a, int b) {
  int total = a + b;
  return total;
}
```

📌 Phân tích:

- `a, b` → **Input**
- `a + b` → **Process**
- `return total` → **Output**

Bước 2: Gọi function trong `main()`

```
void main() {
  int result = calculateTotal(5, 7);
  print(result);
}
```

✅ Kết quả mong đợi

```
12
```

⚠ Lỗi sinh viên hay gặp

❌ Viết function nhưng **không gọi**

```
int calculateTotal(int a, int b) {
  return a + b;
}
```

👉 **Không in gì cả**

💡 **Tip giảng dạy**

Function **không tự chạy**, phải được gọi

5 PHẦN 2 – FUNCTION ≠ UI

📌 Mục tiêu

Tách:

- Logic xử lý
- Hiển thị kết quả

🛠 Thao tác

Viết function chỉ xử lý

```
double calculateAverage(int a, int b) {
  return (a + b) / 2;
}
```


PhucPT10@fpt.edu.vn

Hiển thị ở `main()`

```
void main() {
  double avg = calculateAverage(6, 8);
  print("Average: $avg");
}
```

📌 Giải thích:

- Function **không biết gì về print**
- UI (ở đây là console) quyết định hiển thị

6 PHẦN 3 – CALLBACK: FUNCTION LÀ DỮ LIỆU

📌 Mục tiêu

Hiểu:
Callback = truyền function như tham số

🛠 Thao tác

Bước 1: Function nhận callback

```
void processNumber(int number, Function(int) callback) {
  callback(number);
}
```

Bước 2: Function dùng làm callback

```
void printNumber(int value) {
  print("Value is: $value");
}
```

Bước 3: Gọi callback trong `main()`

```
void main() {
  processNumber(10, printNumber);
}
```

✅ Kết quả

```
Value is: 10
```

📌 Liên hệ Flutter (chưa học UI)

```
onPressed: () {
  // callback
}
```

👉 Flutter **không gọi ngay**,
👉 Chỉ gọi khi có hành động

7 PHẦN 4 – ANONYMOUS FUNCTION (ĐỂ ĐỌC CODE FLUTTER)

📌 Mục tiêu

Không yêu cầu viết phức tạp
Chỉ cần **hiểu và đọc được**

🛠 Ví dụ

```
void process(int number, Function(int) callback) {
  callback(number);
}

void main() {
  process(5, (value) {
    print("Anonymous value: $value");
  });
}
```

📌 Giải thích:

PhucPT10@fpt.edu.vn

- `(value) { ... }` là **anonymous function**
- Không có tên
- Dùng một lần

8 CHECKPOINT TỰ ĐÁNH GIÁ (RẤT QUAN TRỌNG)

Sinh viên **tự trả lời được**:

- Function có chạy nếu không gọi không?
- Input của function nằm ở đâu?
- Output lấy bằng cách nào?
- Callback khác function thường ở điểm nào?
- Nhìn `onPressed: () {}` có hiểu đó là gì không?

👉 Trả lời được → **ĐẠT BÀI**

9 LỖI PHỔ BIẾN & CÁCH SỬA

❌ Lỗi 1: Viết logic + print trong function

Sửa: Function chỉ return

❌ Lỗi 2: Nghĩ rằng viết function là nó chạy

Sửa: Luôn gọi từ `main()`

❌ Lỗi 3: Sợ callback

Sửa: Nhớ câu:
Callback = function truyền như dữ liệu

10 KẾT QUẢ SAU BÀI 1

Sinh viên sẽ:

- ✅ Hiểu function đúng bản chất
- ✅ Không lẫn UI & logic
- ✅ Đọc được code Flutter dùng callback
- ✅ Sẵn sàng học:
  - onPressed
  - onTap
  - Widget callback ở Slot sau

Mở rộng:

- **Bài tập 3: Function + Collection**
- **Bài so sánh "Function junior vs senior"**

PhucPT10@fpt.edu.vn

📝 DỰ ÁN THỰC HÀNH 2
HIỂU ĐÚNG CLASS, OBJECT & CONSTRUCTOR TRONG DART

🎯 MỤC TIÊU BÀI TẬP
Sau bài này, sinh viên **PHẢI nắm chắc**:

- ✅ **Class = bản thiết kế**, object = sản phẩm được tạo từ class
- ✅ Thuộc tính (property) → mô tả **dữ liệu**
- ✅ Method → mô tả **hành vi**
- ✅ Constructor → khởi tạo dữ liệu ban đầu
- ✅ **Widget Flutter thực chất là class Dart**
- ✅ Truyền dữ liệu qua constructor = cách Flutter vận hành UI

⚠️ Nếu không hiểu bài này → sinh viên sẽ **mù mờ khi học Widget, StatelessWidget, truyền tham số**

🧠 TƯ DUY TRỌNG TÂM
“ Flutter không “vẽ UI” bằng magic
Flutter tạo **object từ class Widget ”**
👉 Hiểu Dart Class tốt → học Flutter **nhẹ hơn 50%**

1 TỔNG QUAN BÀI TẬP
📌 Tên bài tập
Student Model – Class & Object Practice
📖 Mô tả
Xây dựng một class Dart để mô tả sinh viên, tạo nhiều object từ class đó, và in thông tin ra console.
👉 **Không UI**
👉 **Không Widget**
👉 **Tập trung vào Dart Class**

2 KIẾN THỨC ĐƯỢC CỦNG CỐ

Kiến thức	Mục đích
Class	Bản thiết kế
Object	Dữ liệu cụ thể
Property	Lưu trạng thái
Method	Hành vi
Constructor	Truyền dữ liệu ban đầu
Truyền tham số	Chuẩn bị cho Widget

3 CHUẨN BỊ (VS CODE – RÕ TỪNG THAO TÁC)
◈ Bước 1: Mở project Flutter
- Mở **VS Code**
- **File → Open Folder**
- Mở project Flutter đang dùng (vd: `flutter_application_2`)

◈ Bước 2: Tạo thư mục `models`
Trong VS Code:
- Chuột phải thư mục `lib`
- Chọn **New Folder**
- Đặt tên: `models`

PhucPT10@fpt.edu.vn

📂 Cấu trúc:

```
lib/
  ├── models/
  └── main.dart
```

◈ Bước 3: Tạo file `student.dart`

- Chuột phải `models`
- **New File**
- Đặt tên: `student.dart`

4 PHẦN 1 – CLASS LÀ BẢN THIẾT KẾ

📌 Mục tiêu

Phân biệt:

- Class (khuôn)
- Object (sản phẩm)

🛠 Viết class `Student`

📄 `lib/models/student.dart`

```
class Student {
  String name;
  double gpa;

  Student(this.name, this.gpa);
}
```

🔍 Phân tích kỹ

Thành phần	Ý nghĩa
`class Student`	Bản thiết kế
`name, gpa`	Thuộc tính (dữ liệu)
`Student(...)`	Constructor

👉 **Chưa có sinh viên nào được tạo ra cả!**

5 PHẦN 2 – OBJECT LÀ SẢN PHẨM TỪ CLASS

📌 Mục tiêu

Hiểu:

Class chỉ là bản mô tả, **object mới là thứ tồn tại thật**

🛠 Thao tác

📄 `lib/main.dart`

```
import 'models/student.dart';

void main() {
  Student s1 = Student("An", 3.6);
  Student s2 = Student("Binh", 3.8);

  print(s1.name);
  print(s2.gpa);
}
```

✅ Kết quả

```
An
3.8
```

📌 Lưu ý:

PhucPT10@fpt.edu.vn

- `Student(...)` → tạo object
- Mỗi object có dữ liệu riêng

6 PHẦN 3 – THUỘC TÍNH & METHOD (DỮ LIỆU vs HÀNH VI)

📌 Mục tiêu

Phân biệt rõ:

- Property → **có gì**
- Method → **làm gì**

🛠 Cập nhật class `Student`

📄 `student.dart`

```
class Student {
  String name;
  double gpa;

  Student(this.name, this.gpa);

  void printInfo() {
    print("$name - GPA: $gpa");
  }
}
```

🛠 Gọi method từ `main()`

```
void main() {
  Student s1 = Student("An", 3.6);
  Student s2 = Student("Binh", 3.8);

  s1.printInfo();
  s2.printInfo();
}
```

✅ Kết quả

```
An - GPA: 3.6
Binh - GPA: 3.8
```

7 PHẦN 4 – CONSTRUCTOR = KHỞI TẠO DỮ LIỆU BAN ĐẦU

📌 Mục tiêu

Hiểu:

Constructor chạy **NGAY KHI object được tạo**

🛠 Phân tích constructor

```
Student(this.name, this.gpa);
```

👉 Tương đương:

```
Student(String name, double gpa) {
  this.name = name;
  this.gpa = gpa;
}
```

📌 Constructor:

- Không return
- Tên trùng class

8 PHẦN 5 – LIÊN HỆ TRỰC TIẾP VỚI FLUTTER

📌 Tư duy then chốt

```
class MyWidget extends StatelessWidget {
  final String title;
```


PhucPT10@fpt.edu.vn

```
  MyWidget(this.title);
}
```

👉 **Flutter Widget = Dart class**
👉 **Truyền dữ liệu qua constructor**
👉 **Build UI từ dữ liệu**

📌 So sánh dễ hiểu

Dart Core	Flutter
`Student`	`Widget`
`name, gpa`	dữ liệu hiển thị
constructor	truyền dữ liệu
object	widget instance

9 CHECKPOINT TỰ ĐÁNH GIÁ

Sinh viên **phải trả lời được**:

- Class có phải là dữ liệu thật không?
- Khi nào constructor chạy?
- Object khác nhau ở điểm nào?
- Widget Flutter có phải class Dart không?
- Vì sao Flutter truyền dữ liệu qua constructor?

👉 Trả lời được → **ĐÃ SẴN SÀNG HỌC WIDGET**

10 LỖI PHỔ BIẾN & CÁCH SỬA

❌ Lỗi 1: Viết logic trong `main()`

Sửa: Đưa hành vi vào method của class

❌ Lỗi 2: Class quá nhiều trách nhiệm

Sửa: Class mô tả **một đối tượng rõ ràng**

❌ Lỗi 3: Không dùng constructor

Sửa: Luôn khởi tạo dữ liệu từ đầu

11 KẾT QUẢ SAU BÀI 2

Sinh viên sẽ:

- ✅ Hiểu đúng bản chất class & object
- ✅ Không sợ Widget Flutter
- ✅ Hiểu truyền dữ liệu qua constructor
- ✅ Có nền tảng vững để học:
  - StatelessWidget
  - UI nhận dữ liệu

PhucPT10@fpt.edu.vn

DỰ ÁN THỰC HÀNH 3

COLLECTIONS: LIST & MAP

MỤC TIÊU BÀI TẬP

Sau bài này, sinh viên **phải hiểu sâu**:

- ✅ **List = danh sách có thứ tự**
- ✅ **Map = dữ liệu dạng key – value (có nhãn)**
- ✅ Flutter **KHÔNG vẽ UI thủ công**, mà:
  Dữ liệu → duyệt → biến thành widget
- ✅ Biết cách:
  - Lưu nhiều object trong List
  - Duyệt dữ liệu bằng `for`, `for-in`
  - Chuẩn bị dữ liệu để render UI Flutter (mô phỏng)

TƯ DUY CỐT LÕI

❝ Flutter không hiển thị từng dòng
Flutter hiển thị **mỗi phần tử trong Collection** ❞

👉 Hiểu Collection → học `ListView.builder` **rất nhẹ**

1 TỔNG QUAN BÀI TẬP

Tên bài tập

Student List Processor

Mô tả

- Quản lý danh sách sinh viên bằng `List<Student>`
- Duyệt danh sách để:
  - In toàn bộ sinh viên
  - Lọc sinh viên GPA cao
- Mô phỏng quá trình:
  Dữ liệu → duyệt → hiển thị

2 CHUẨN BỊ PROJECT

Cấu trúc thư mục (giữ nguyên)

```
lib/
 ├── models/
 │    └── student.dart
 └── main.dart
```

⚠️ Nếu chưa có `Student` → quay lại **Bài 2**

3 PHẦN 1 – LIST: DANH SÁCH CÓ THỨ TỰ

Mục tiêu

Hiểu:

- List lưu **nhiều object**
- Có **thứ tự**
- Truy cập bằng index

Bước 1: Tạo List<Student>

`lib/main.dart`

```
import 'models/student.dart';
```


PhucPT10@fpt.edu.vn

```
void main() {
  List<Student> students = [
    Student("An", 3.6),
    Student("Binh", 3.8),
    Student("Dung", 3.5),
  ];

  print(students.length);
}
```

Giải thích

Thành phần	Ý nghĩa
`List<Student>`	Danh sách sinh viên
`[ ... ]`	Khởi tạo list
`length`	Số phần tử

List có thứ tự:

- `students[0]` → An
- `students[1]` → Binh

Lỗi sinh viên hay gặp

- Nghĩ List giống biến đơn
- Không hiểu index bắt đầu từ **0**

4 PHẦN 2 – DUYỆT LIST BẰNG for / for-in

Mục tiêu

Hiểu:
UI Flutter hiển thị **từng phần tử trong List**

Duyệt bằng `for`

```
for (int i = 0; i < students.length; i++) {
  print(students[i].name);
}
```

Dùng khi:

- Cần index
- Cần kiểm soát chặt

Duyệt bằng `for-in`

```
for (var student in students) {
  student.printInfo();
}
```

Dùng khi:

- Đọc dễ
- Không cần index

Output

```
An - GPA: 3.6
Binh - GPA: 3.8
Dung - GPA: 3.5
```

5 PHẦN 3 – MAP: DỮ LIỆU CÓ NHÃN (KEY – VALUE)

PhucPT10@fpt.edu.vn

📌 Mục tiêu

Hiểu:

- Map dùng khi dữ liệu **có ý nghĩa gắn nhãn**
- Truy xuất bằng key, không phải index

🛠 Tạo Map sinh viên

```
Map<String, Student> studentMap = {
  "s1": Student("An", 3.6),
  "s2": Student("Binh", 3.8),
  "s3": Student("Dung", 3.5),
};
```

🔍 Phân tích

Thành phần	Ý nghĩa
`"s1"`	key
`Student(...)`	value
`Map<String, Student>`	Map

📌 Không có thứ tự cố định như List

🛠 Truy xuất dữ liệu

```
studentMap["s2"]?.printInfo();
```

📌 Dấu ?:

- Tránh lỗi khi key không tồn tại

6 PHẦN 4 – LIÊN HỆ TRỰC TIẾP VỚI FLUTTER UI

📌 Tư duy cực kỳ quan trọng

Trong Flutter:

```
ListView(
  children: students.map((s) {
    return Text(s.name);
  }).toList(),
);
```

👉 **students (List dữ liệu)**
→ **map() (duyệt)**
→ **Text (Widget)**

🧠 Mô phỏng bằng Dart Core (không UI)

```
List<String> displayData = [];

for (var student in students) {
  displayData.add("${student.name} - GPA: ${student.gpa}");
}

print(displayData);
```

✅ Output

```
[An - GPA: 3.6, Binh - GPA: 3.8, Dung - GPA: 3.5]
```

📌 Đây chính là **bước chuẩn bị dữ liệu cho UI**

7 PHẦN 5 – SAI LẦM PHỔ BIẾN

❌ Lỗi 1: Xử lý dữ liệu trong UI

Hậu quả: code rối, khó sửa
Giải pháp: xử lý trong Dart trước

PhucPT10@fpt.edu.vn

❌ Lỗi 2: Dùng Map khi cần List

Hậu quả: duyệt khó
Giải pháp: UI hiển thị → ưu tiên List

❌ Lỗi 3: Không hiểu map() trả về gì

Nhớ: `map()` → **List mới**, không sửa List cũ

8 CHECKPOINT TỰ ĐÁNH GIÁ

Sinh viên trả lời được:

- List khác Map ở điểm nào?
- Khi nào nên dùng List?
- UI Flutter thường render từ đâu?
- Vì sao phải chuẩn bị dữ liệu trước UI?

9 KẾT QUẢ SAU BÀI 3

Sinh viên:

- ✅ Không còn sợ Collection
- ✅ Hiểu UI Flutter dựa trên List
- ✅ Biết duyệt & chuẩn bị dữ liệu
- ✅ Sẵn sàng học:
  - `ListView`
  - `Column(children: ...)`

PhucPT10@fpt.edu.vn

MỤC TIÊU

Sau bài này, sinh viên **bắt buộc phải làm được**:

- ✅ Hiểu **bản chất `map()` và `where()`**
- ✅ Phân biệt rõ:
  - `for` / `for-in` → **làm từng bước**
  - `map()` / `where()` → **biến đổi dữ liệu**
- ✅ Chuẩn bị dữ liệu **TRƯỚC UI Flutter**
- ✅ Đọc được code Flutter dùng `map()`

⚠️ **Chưa yêu cầu viết code "ngắn cho đẹp"**

👉 Ưu tiên **đúng – rõ – dễ debug**

TƯ DUY CỐT LÕI (CỰC QUAN TRỌNG)

" UI Flutter KHÔNG xử lý logic
UI Flutter CHỈ NHẬN DỮ LIỆU ĐÃ CHUẨN BỊ **"**

1 ÔN NHANH BỐI CẢNH

Ta đã có:

📄 `student.dart`

```
class Student {
  String name;
  double gpa;

  Student(this.name, this.gpa);

  void printInfo() {
    print("$name - GPA: $gpa");
  }
}
```

📄 `main.dart`

```
List<Student> students = [
  Student("An", 3.6),
  Student("Binh", 3.8),
  Student("Dung", 3.5),
];
```

2 PHẦN A – where(): LỌC DỮ LIỆU

Mục tiêu

Hiểu:

- `where()` **lọc dữ liệu**
- Trả về **List mới**
- Không làm thay đổi List gốc

Ví dụ: Lọc sinh viên GPA ≥ 3.7

```
List<Student> excellentStudents =
    students.where((student) {
      return student.gpa >= 3.7;
    }).toList();
```


PhucPT10@fpt.edu.vn

🔍 Giải thích

Thành phần	Ý nghĩa
`where()`	Lọc dữ liệu
`(student)`	Từng phần tử
`return true`	Giữ lại
`toList()`	Chuyển thành List

📌 **Nếu không gọi `toList()` → kết quả KHÔNG phải List**

🧪 Kiểm tra kết quả

```
for (var s in excellentStudents) {
  s.printInfo();
}
```

Output

```
Binh - GPA: 3.8
```

❌ Lỗi sinh viên hay gặp

- Quên `toList()`
- Nghĩ `where()` sửa List cũ

3 PHẦN B – map(): BIẾN ĐỔI DỮ LIỆU

📌 Mục tiêu

Hiểu:

- `map()` dùng để **chuyển dữ liệu từ dạng A → dạng B**
- Thường dùng để:
  - Chuẩn bị text
  - Chuẩn bị widget (sau này)

🛠 Ví dụ: Student → String

```
List<String> displayTexts =
    students.map((student) {
      return "${student.name} - GPA: ${student.gpa}";
    }).toList();
```

🔍 Phân tích

map()	where()
Biến đổi	Lọc
A → B	A → A
Luôn tạo dữ liệu mới	Luôn tạo List mới

🧪 Kiểm tra

```
print(displayTexts);
```

Output

```
[An - GPA: 3.6, Binh - GPA: 3.8, Dung - GPA: 3.5]
```

4 PHẦN C – KẾT HỢP where() + map()

📌 Mục tiêu

Chuẩn bị **dữ liệu đúng cho UI**, không xử lý trong UI

PhucPT10@fpt.edu.vn

🛠 Ví dụ thực tế

UI chỉ hiển thị sinh viên GPA ≥ 3.6

```
List<String> uiData =
    students
        .where((s) => s.gpa >= 3.6)
        .map((s) => "${s.name} (${s.gpa})")
        .toList();
```

🧠 Tư duy

```
Dữ liệu gốc
   ↓ where()
Dữ liệu đúng
   ↓ map()
Dữ liệu hiển thị
```

🧪 Output

```
[An (3.6), Binh (3.8)]
```

5 PHẦN D – SO SÁNH VỚI for (RẤT QUAN TRỌNG)

🛠 Viết bằng for

```
List<String> uiData = [];

for (var s in students) {
  if (s.gpa >= 3.6) {
    uiData.add("${s.name} (${s.gpa})");
  }
}
```

📊 So sánh

for	map/where
Dễ debug	Ngắn gọn
Rõ từng bước	Dễ rối nếu lạm dụng
Tốt cho beginner	Tốt khi đã quen

📌 **Nguyên tắc dạy sinh viên**

👉 Hiểu `for` TRƯỚC → dùng `map()` SAU

6 LIÊN HỆ TRỰC TIẾP VỚI FLUTTER

📌 Code Flutter (chỉ để đọc)

```
Column(
  children: students
      .where((s) => s.gpa >= 3.6)
      .map((s) => Text(s.name))
      .toList(),
);
```

📌 UI **KHÔNG xử lý logic**

- Logic đã làm xong trong Dart

7 SAI LẦM PHỔ BIẾN (RẤT HAY GẶP)

❌ Viết where/map trong build()

➡️ build chạy lại liên tục → tốn tài nguyên

PhucPT10@fpt.edu.vn

❌ Code quá "functional" để khoe
➡️ Không ai đọc nổi khi debug

❌ Không hiểu map() trả về gì
➡️ Dẫn tới lỗi kiểu dữ liệu

8 CHECKPOINT TỰ KIỂM TRA

Sinh viên trả lời được:

- where() trả về gì?
- map() dùng khi nào?
- Vì sao cần `toList()`?
- Vì sao UI không nên xử lý logic?

9 KẾT QUẢ SAU BÀI 4

Sinh viên:

- ✅ Hiểu sâu `map()` & `where()`
- ✅ Chuẩn bị dữ liệu cho UI Flutter
- ✅ Đọc được code Flutter thật
- ✅ Không còn sợ functional style

PhucPT10@fpt.edu.vn

1 TỔNG QUAN DỰ ÁN

📌 Tên dự án

Student Data Processor (Logic-only Mini Project)

🔑 Mô tả dự án

Dự án mô phỏng **xử lý dữ liệu sinh viên** trong một app Flutter:

- Quản lý danh sách sinh viên (data)
- Xử lý logic:
  - Thêm / cập nhật / lọc sinh viên
  - Chuẩn bị dữ liệu để hiển thị UI (mô phỏng)
- **Không tập trung UI**, chỉ dùng Flutter để:
  - Chạy project
  - In log kiểm tra logic

👉 Đây là **bài toán rất gần với project Flutter thật**, nhưng được **giảm tải UI** để sinh viên tập trung Dart Core.

🎯 Mục tiêu học tập

Sau mini project này, sinh viên:

- Hiểu **Dart được dùng thế nào trong Flutter**
- Biết:
  - Tách logic khỏi "UI"
  - Dùng function đúng vai trò
  - Dùng class để quản lý dữ liệu
  - Xử lý List / Map có tư duy
- Chuẩn bị tốt cho:
  - Slot UI & Widget
  - Project Flutter lớn hơn

👥 Đối tượng phù hợp

- Sinh viên **đã học xong Slot 3**
- Trình độ **trung bình – yếu**, cần củng cố nền

2 BÀI TOÁN & CHỨC NĂNG LOGIC CHÍNH

📄 Bài toán

Quản lý **danh sách sinh viên**, mỗi sinh viên có:

- id
- name
- age
- gpa
- major

◈ Các chức năng logic chính

1. Thêm sinh viên

- Áp dụng:
  - Class + constructor
  - Function xử lý logic
- Độ phức tạp: ⭐⭐

2. Cập nhật GPA sinh viên

- Áp dụng:
  - Function có tham số
  - for / for-in

PhucPT10@fpt.edu.vn

- Độ phức tạp: ⭐⭐

3. Lọc sinh viên GPA cao

- Áp dụng:
  - List
  - where()
- Độ phức tạp: ⭐⭐

4. Chuẩn bị dữ liệu hiển thị (mô phỏng UI)

- Áp dụng:
  - map()
  - Callback function
- Độ phức tạp: ⭐⭐⭐

👉 **Mỗi chức năng đều gắn trực tiếp với Dart Core Slot 3**

3 CÔNG NGHỆ & CÔNG CỤ SỬ DỤNG

- Dart
- Flutter SDK (chỉ để chạy)
- Android Studio / VS Code
- Debug bằng:
  - `print()`
  - Console log

⚠️ **Không yêu cầu UI, layout, widget**

4 CẤU TRÚC THƯ MỤC DỰ ÁN (TẬP TRUNG DART)

```
lib/
│
├── models/
│   └── student.dart
│
├── services/
│   └── student_service.dart
│
├── data/
│   └── sample_data.dart
│
└── main.dart
```

🎯 Giải thích sư phạm

- `models/` → định nghĩa **dữ liệu**
- `services/` → xử lý **logic**
- `data/` → dữ liệu mẫu để test
- `main.dart` → nơi chạy & kiểm tra

👉 Đây là **cách tổ chức tối thiểu nhưng rất quan trọng cho project thật**

5 DANH SÁCH CÁC BƯỚC THỰC HIỆN (TÓM TẮT)

1. Tạo Flutter project
2. Tạo class `Student`
3. Tạo dữ liệu mẫu
4. Viết function thêm sinh viên
5. Viết function cập nhật GPA
6. Viết function lọc sinh viên
7. Chuẩn bị dữ liệu hiển thị (map)
8. Kiểm tra logic bằng log

PhucPT10@fpt.edu.vn

6 HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC

BƯỚC 1 – Tạo Flutter project bằng VS Code

Mục tiêu

- Có một project Flutter **để chạy Dart code**
- Chưa quan tâm UI

Thao tác chi tiết trong VS Code

1. **Mở VS Code**
2. Nhấn tổ hợp phím:
   - `Ctrl + Shift + P` (Windows)
3. Gõ:
   `Flutter: New Project`
4. Chọn:
   `Flutter Application`
5. Chọn thư mục lưu project (ví dụ: Desktop hoặc Documents)
6. Nhập tên project:
   `student_data_processor`
   ⚠ Lưu ý:
   - Không dùng dấu
   - Không viết hoa
7. Nhấn **Enter** → chờ VS Code tạo project

Kiểm tra thành công

- VS Code mở project
- Thấy cấu trúc:
- `lib/`
- `test/`
- `pubspec.yaml`

BƯỚC 2 – Mở thư mục `lib` & làm quen giao diện

Mục tiêu

- Biết **code Dart nằm ở đâu**
- Không sửa nhầm file

Thao tác

1. Trong **Explorer (bên trái VS Code)**
2. Mở thư mục:
3. `lib`
4. Click vào file:
5. `main.dart`

👉 Đây là **file Dart chính**, nơi app bắt đầu chạy.

⚠ **Chưa cần hiểu UI trong file này**

BƯỚC 3 – Tạo các thư mục Dart cần thiết

Mục tiêu

- Tập thói quen **tổ chức code rõ ràng ngay từ đầu**

Thao tác

Trong thư mục `lib`:

1. **Chuột phải → New Folder**
2. Tạo lần lượt các thư mục:

PhucPT10@fpt.edu.vn

```
models
services
data
```

👉 Kết quả:

```
lib/
├── models/
├── services/
├── data/
└── main.dart
```

💡 Ghi nhớ:

- `models` → dữ liệu
- `services` → logic xử lý
- `data` → dữ liệu mẫu để test

🔷 BƯỚC 4 – Tạo class Student (model dữ liệu)

📌 Mục tiêu

- Biết cách:
  - Tạo file Dart
  - Viết class
  - Dùng constructor

🛠 Thao tác trong VS Code

1. Chuột phải vào thư mục:
   `lib/models`
2. Chọn:
   `New File`
3. Đặt tên file:
   `student.dart`
4. Mở file `student.dart`
5. Gõ code sau:

```dart
class Student {
  final int id;
  final String name;
  final int age;
  double gpa;
  final String major;

  Student({
    required this.id,
    required this.name,
    required this.age,
    required this.gpa,
    required this.major,
  });
}
```

🧠 Giải thích cho sinh viên mới

- `class Student` → khuôn mẫu sinh viên
- Constructor dùng để:
  - Tạo object
  - Truyền dữ liệu **gọn gàng – an toàn**

⚠️ **Chưa cần hiểu `required` sâu**, chỉ cần nhớ:
Constructor giúp truyền dữ liệu đúng chỗ

PhucPT10@fpt.edu.vn

◈ BƯỚC 5 – Tạo dữ liệu mẫu để test logic

📌 Mục tiêu

- Có dữ liệu sẵn để xử lý List / map / where

🛠 Thao tác

1. Chuột phải vào:
2. `lib/data`
3. Chọn **New File**
4. Đặt tên:
5. `sample_data.dart`
6. Viết code:

```
import '../models/student.dart';

List<Student> sampleStudents = [
  Student(id: 1, name: 'An', age: 20, gpa: 3.2, major: 'IT'),
  Student(id: 2, name: 'Binh', age: 21, gpa: 3.8, major: 'Business'),
  Student(id: 3, name: 'Chi', age: 19, gpa: 2.9, major: 'IT'),
];
```

⚠ Lỗi sinh viên hay gặp

- Quên `import`
- Gõ sai đường dẫn

👉 Nếu VS Code gạch đỏ:

- Đưa chuột vào → xem gợi ý → sửa theo VS Code

◈ BƯỚC 6 – Viết function xử lý logic (service)

📌 Mục tiêu

- Tách logic ra khỏi UI
- Dùng function đúng vai trò

🛠 Thao tác

1. Chuột phải vào:
2. `lib/services`
3. Tạo file:
4. `student_service.dart`

✏ Viết function thêm sinh viên

```
import '../models/student.dart';

void addStudent(List<Student> list, Student student) {
  list.add(student);
}
```

💡 Ghi nhớ cho sinh viên:
Function chỉ làm **1 việc rõ ràng**

✏ Viết function cập nhật GPA

```
void updateGpa(
  List<Student> list,
  int studentId,
  double newGpa,
) {
  for (var student in list) {
    if (student.id == studentId) {
      student.gpa = newGpa;
      break;
```


PhucPT10@fpt.edu.vn

```
    }
  }
}
```

```
Viết function lọc sinh viên GPA cao
List<Student> filterHighGpa(
  List<Student> list,
  double minGpa,
) {
  return list.where((s) => s.gpa >= minGpa).toList();
}
```

BƯỚC 7 – Chuẩn bị dữ liệu hiển thị (map + callback)

Mục tiêu

- Hiểu callback
- Chuẩn bị dữ liệu **trước UI**
  lib/services/student_service.dart

```
List<String> prepareDisplayData(
  List<Student> list,
  String Function(Student) formatter,
) {
  return list.map(formatter).toList();
}
```

Giải thích dễ hiểu:

- `formatter` = "cách hiển thị"
- UI sau này chỉ việc dùng kết quả

BƯỚC 8 – Test toàn bộ logic trong `main.dart`

Mục tiêu

- Kiểm tra logic **không cần UI**

Thao tác

1. Mở `lib/main.dart`
2. **Xóa bớt code UI**, chỉ giữ `main()`
3. Import các file cần dùng:

```
import 'data/sample_data.dart';
import 'models/student.dart';
import 'services/student_service.dart';
```

Viết code test

```
void main() {
  final students = [...sampleStudents];

  addStudent(
    students,
    Student(id: 4, name: 'Dung', age: 22, gpa: 3.5, major: 'Design'),
  );

  updateGpa(students, 1, 3.6);

  final topStudents = filterHighGpa(students, 3.5);

  final display = prepareDisplayData(
    topStudents,
    (s) => '${s.name} - GPA: ${s.gpa}',
  );

  for (var line in display) {
```


PhucPT10@fpt.edu.vn

```
    print(line);
  }
}
```

🔷 BƯỚC 9 – Chạy project & xem kết quả

🛠️ Chạy bằng VS Code

1. Mở **Terminal trong VS Code**
   - Menu: `Terminal → New Terminal`
2. Gõ:
3. `flutter run`
4. Hoặc nhấn nút **Run** ▶️

✅ Kiểm tra kết quả

- Console hiển thị:

```
Building Windows application...                                    53.1s
√ Built build\windows\x64\runner\Debug\flutter_application_3.exe
An - GPA: 3.6
Binh - GPA: 3.8
Dung - GPA: 3.5
Syncing files to device Windows...                                 966ms

Flutter run key commands.
r Hot reload.
```

👉 Nếu ra đúng → **logic OK**

🎓 CHECKPOINT CHO SINH VIÊN

Nếu sinh viên:

- Biết tạo file / folder trong VS Code
- Biết import đúng
- Biết test logic bằng `print()`
- Không viết logic trong UI

➡️ **Sinh viên đã vượt qua rào cản lớn nhất khi học Flutter**

PhucPT10@fpt.edu.vn

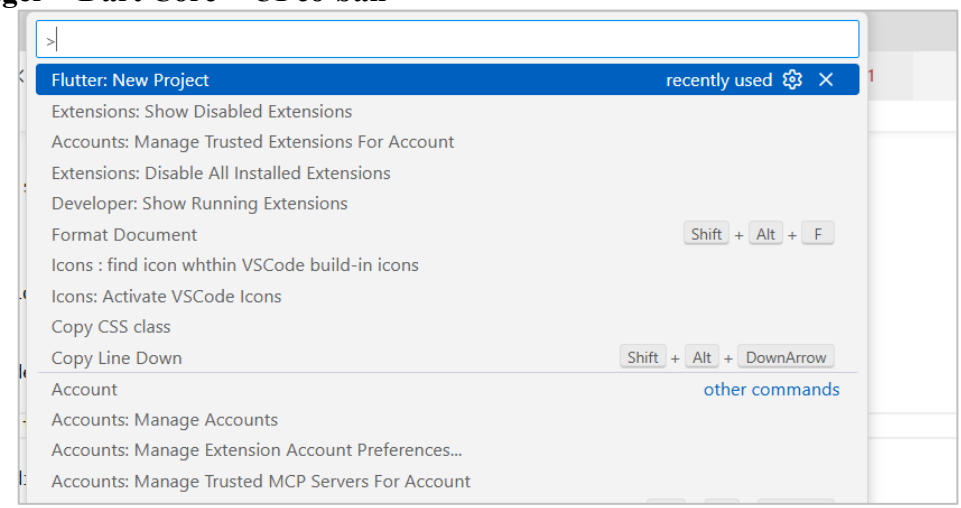
BÀI TẬP TỔNG HỢP 2

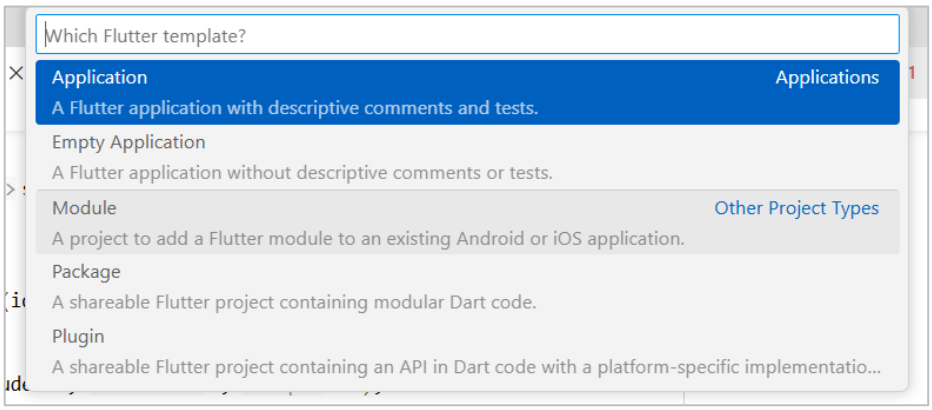
STUDENT MANAGER – LOGIC FIRST, UI LATER

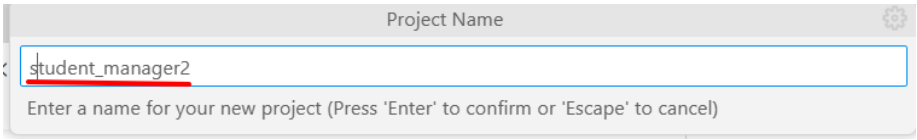
1 TỔNG QUAN DỰ ÁN

Tên dự án

Student Manager – Dart Core + UI cơ bản







Mô tả

Ứng dụng Flutter **hiển thị danh sách sinh viên** và cho phép:

- Lọc sinh viên theo GPA
- Hiển thị dữ liệu đã được xử lý từ Dart logic

⚠️ UI **đơn giản**, không focus layout

👉 Mục tiêu là **thấy Dart Core hoạt động trong Flutter thật**

Mục tiêu học tập

Sinh viên:

- Hiểu rõ **Dart xử lý logic ở đâu**
- Biết **truyền dữ liệu từ Dart → UI**
- Áp dụng:
  - Function
  - Class & constructor
  - List, map(), where()

PhucPT10@fpt.edu.vn

Đối tượng

- Sinh viên đã học xong Slot 3
- Nền tảng Flutter cơ bản

2 BÀI TOÁN & CHỨC NĂNG LOGIC

Bài toán

Quản lý danh sách sinh viên và hiển thị lên UI Flutter

Chức năng logic

Chức năng	Dart Core sử dụng
Tạo sinh viên	Class + constructor
Lưu danh sách	List
Lọc GPA	where()
Chuẩn bị text hiển thị	map()
Xử lý logic	function
UI phản hồi	callback

3 CÔNG NGHỆ & CÔNG CỤ

- Flutter SDK
- Dart
- VS Code
- Emulator hoặc Windows app
- Debug bằng `print()`

4 CẤU TRÚC THƯ MỤC (RẤT QUAN TRỌNG)

```
lib/
├── models/
│   └── student.dart
├── services/
│   └── student_service.dart
└── main.dart
```

Giải thích

- `models/` → mô tả dữ liệu
- `services/` → xử lý logic
- `main.dart` → UI + gọi logic

👉 **Đây là cách tổ chức chuẩn cho Flutter thật**

5 DANH SÁCH CÁC BƯỚC THỰC HIỆN (TÓM TẮT)

1. Tạo project Flutter
2. Tạo model `Student`
3. Tạo service xử lý logic
4. Viết function lọc & chuẩn bị dữ liệu
5. Tạo UI đơn giản
6. Kết nối UI ↔ Dart logic
7. Chạy & kiểm tra kết quả

6 HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC (QUAN TRỌNG NHẤT)

BƯỚC 1 – TẠO PROJECT FLUTTER (VS CODE)

Mục tiêu

Có project Flutter để bắt đầu

PhucPT10@fpt.edu.vn

🛠 Thao tác

1. Mở **VS Code**
2. `Ctrl + Shift + P`
3. Chọn **Flutter: New Project**
4. Chọn **Application**
5. Đặt tên: `student_manager`
6. Chọn thư mục lưu
7. Chờ Flutter tạo project

✅ Kiểm tra

- Chạy `flutter run`
- App mở lên (có thể là màn hình trắng)

BƯỚC 2 – TẠO CLASS MODEL (student.dart)

📌 Mục tiêu

Hiểu:

- Class là bản thiết kế
- Constructor khởi tạo dữ liệu

🛠 Thao tác

1. Chuột phải `lib/` → **New Folder** → `models`
2. Trong `models` → **New File** → `student.dart`

📋 Code

```
class Student {
  String name;
  double gpa;

  Student(this.name, this.gpa);

  String display() {
    return "$name - GPA: $gpa";
  }
}
```

⚠ Lỗi thường gặp

- Nhầm constructor với function
- Quên kiểu dữ liệu

BƯỚC 3 – TẠO SERVICE XỬ LÝ LOGIC

📌 Mục tiêu

Tách logic khỏi UI

🛠 Thao tác

1. Tạo thư mục `services`
2. Tạo file `student_service.dart`

📋 Code

```
import '../models/student.dart';

class StudentService {
  List<Student> students = [
    Student("An", 3.6),
    Student("Binh", 3.8),
    Student("Dung", 3.4),
  ];

  List<Student> getExcellentStudents() {
    return students.where((s) => s.gpa >= 3.6).toList();
  }

  List<String> getDisplayTexts() {
```


PhucPT10@fpt.edu.vn

```
    return getExcellentStudents()
        .map((s) => s.display())
        .toList();
  }
}
```

Giải thích

- `where()` → lọc
- `map()` → biến đổi
- UI **KHÔNG làm việc này**

BƯỚC 4 – KẾT NỐI UI VỚI LOGIC (main.dart)

Mục tiêu

Thấy Dart Core hoạt động trong Flutter

Thao tác

Mở `lib/main.dart` và thay bằng:

```
import 'package:flutter/material.dart';
import 'services/student_service.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    final service = StudentService();
    final data = service.getDisplayTexts();

    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: const Text("Student Manager")),
        body: ListView(
          children: data.map((text) {
            return ListTile(title: Text(text));
          }).toList(),
        ),
      ),
    );
  }
}
```

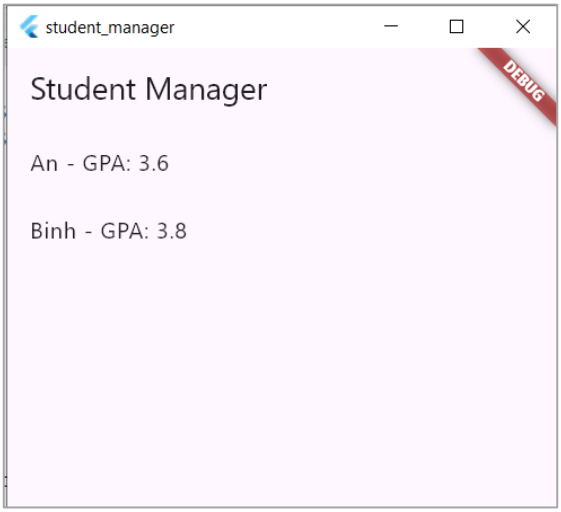
BƯỚC 5 – CHẠY & KIỂM TRA

Kết quả mong đợi

UI hiển thị:

```
PS D:\NodeJS\SP26\slot3\Student Manager\student_manager> flutter run
Connected devices:
Windows (desktop) • windows • windows-x64    • Microsoft Windows [Version 10.0.19045.6456]
Chrome (web)      • chrome  • web-javascript • Google Chrome 143.0.7499.170
Edge (web)        • edge    • web-javascript • Microsoft Edge 143.0.3650.96
[1]: Windows (windows)
[2]: Chrome (chrome)
[3]: Edge (edge)
Please choose one (or "q" to quit): 1
Launching lib\main.dart on Windows in debug mode...
Building Windows application...
```


PhucPT10@fpt.edu.vn



📌 Checkpoint cho sinh viên

- UI không xử lý logic
- Logic nằm trong service
- map() dùng để tạo widget

7 KIỂM TRA & ĐÁNH GIÁ LOGIC

Gợi ý test:

- Đổi GPA
- Thêm sinh viên
- Đổi điều kiện lọc

8 LỖI PHỔ BIẾN & CÁCH SỬA

Lỗi	Nguyên nhân	Cách sửa
Logic trong build()	Chưa tách service	Đưa sang class riêng
Code map dài	Tham lam	Tách function
UI khó đọc	Xử lý dữ liệu trong UI	Chuẩn bị trước

9 KẾT QUẢ SAU DỰ ÁN

Sinh viên:

- ✅ Hiểu Dart Core dùng thế nào trong Flutter
- ✅ Không còn viết logic lung tung trong UI
- ✅ Sẵn sàng học:
  - Widget
  - State
  - ListView.builder

10 BÀI TẬP MỞ RỘNG

1. Thêm filter GPA ≥ 3.5
2. Thêm named constructor
3. Refactor service
4. Tách widget hiển thị

PhucPT10@fpt.edu.vn

BÀI TẬP TỔNG HỢP 3
QUICK SHOP MVP (Mô phỏng Giỏ hàng)

1 TỔNG QUAN DỰ ÁN

- **Tên dự án:** Quick Shop Logic Demo.
- **Mô tả:** Một ứng dụng đơn giản hiển thị danh sách sản phẩm. Người dùng có thể xem danh sách, lọc sản phẩm theo giá và bấm vào sản phẩm để "thêm vào giỏ" (mô phỏng bằng in log).
- **Mục tiêu học tập:**
  - Hiểu cách tạo **Class** để định nghĩa dữ liệu sản phẩm.
  - Sử dụng **List** và hàm **.map()** để biến dữ liệu thô thành giao diện (Widget).
  - Sử dụng **Function & Callback** để xử lý sự kiện khi người dùng bấm nút.
- **Đối tượng:** Sinh viên mới học xong Slot 3, chưa cần biết nhiều về State Management.

2 BÀI TOÁN & CHỨC NĂNG LOGIC CHÍNH

Chúng ta sẽ tập trung vào phần "chìm" của tảng băng trôi (Dart Logic):

1. **Mô hình hóa dữ liệu (Classes):** Tạo khuôn mẫu cho một Sản phẩm (Product).
2. **Xử lý danh sách (Collections):**
   - Lưu trữ danh sách sản phẩm gốc.
   - Lọc các sản phẩm có giá < 50$ (dùng .where).
   - Chuyển đổi danh sách sản phẩm thành danh sách Widget để hiển thị (dùng .map).
3. **Tương tác (Functions):**
   - Viết hàm xử lý khi người dùng bấm vào sản phẩm.
   - Truyền hàm này dưới dạng tham số (Callback).

3 CÔNG NGHỆ & CÔNG CỤ

- **Ngôn ngữ:** Dart (phiên bản mới nhất).
- **Framework:** Flutter (để chạy giao diện).
- **IDE:** VS Code hoặc Android Studio.
- **Công cụ debug:** Debug Console (quan trọng để xem print).

4 CẤU TRÚC THƯ MỤC DỰ ÁN

Để tập thói quen tốt ("Clean Code"), chúng ta không viết tất cả vào 1 file.

Plaintext

```
lib/
├── main.dart            # CHỈ UI + gọi logic
├── models/
│   └── product.dart     # Model dữ liệu
├── data/
│   └── data_source.dart # Dữ liệu giả lập
├── services/
│   └── product_service.dart # ⭐ LOGIC XỬ LÝ (RẤT QUAN TRỌNG)
```

Tại sao làm vậy?

- models: Để sau này muốn sửa thông tin sản phẩm (thêm màu sắc, kích thước) thì chỉ cần sửa file này.
- data: Tách biệt dữ liệu ra khỏi giao diện. UI không nên chứa dữ liệu cứng (Hardcode).

Thành phần	Vai trò
`models/`	Dữ liệu là gì
`data/`	Dữ liệu đến từ đâu
`services/`	Xử lý dữ liệu ra sao
`main.dart`	Hiển thị kết quả

PhucPT10@fpt.edu.vn

5 TÓM TẮT CÁC BƯỚC THỰC HIỆN

1. **Khởi tạo:** Tạo project Flutter mới & làm sạch code cũ.
2. **Tạo Model:** Xây dựng Class Product với Constructor chuẩn.
3. **Tạo Data:** Tạo List<Product> giả lập.
4. **Xử lý UI cơ bản:** Dựng khung màn hình Scaffold.
5. **Logic cốt lõi (Core):** Dùng .map() để biến đổi List data thành List Widget.
6. **Logic tương tác:** Viết Function xử lý sự kiện onTap.
7. **Logic nâng cao:** Dùng .where() để lọc dữ liệu trước khi hiển thị.

6 HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC (STEP-BY-STEP)

BƯỚC 1: Khởi tạo Project & Dọn dẹp

- **Mục tiêu:** Có một trang giấy trắng.
- **Thao tác:**
    1. Tạo project: flutter create quick_shop_logic
    2. Mở lib/main.dart, xóa hết, chỉ giữ lại khung cơ bản sau:

Dart
```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MaterialApp(home: QuickShopPage()));
}

// Chúng ta dùng Stateless vì chưa học State Management
class QuickShopPage extends StatelessWidget {
  const QuickShopPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Quick Shop Dart Logic")),
      body: const Center(child: Text("Chuẩn bị code...")),
    );
  }
}
```

BƯỚC 2: Tạo Model (Class Product)

- **Mục tiêu:** Áp dụng kiến thức **Classes & Constructors**.
- **Thao tác:** Tạo file lib/models/product.dart.
- **Code:**

Dart
```dart
// lib/models/product.dart
class Product {
  // 1. Thuộc tính (Properties) - Dùng final vì dữ liệu không đổi sau khi tạo
  final String id;
  final String name;
  final double price;

  // 2. Constructor - Dùng Named Parameters và required
  Product({
    required this.id,
    required this.name,
    required this.price,
  });
}
```

💡 **Tại sao?** Dùng Named Parameters ({}) giúp khi khởi tạo object ta biết rõ tham số nào là tên, tham số nào là giá, tránh nhầm lẫn.

BƯỚC 3: Tạo Dữ Liệu Giả (Collection)

- **Mục tiêu:** Áp dụng kiến thức **List**.
- **Thao tác:** Tạo file lib/data/data_source.dart.
- **Code:**

Dart

```
// lib/data/data_source.dart
import '../models/product.dart'; // Nhớ import file model

// Tạo một List chứa các object Product
List<Product> rawProducts = [
  Product(id: "p1", name: "Laptop Gaming", price: 1200.0),
  Product(id: "p2", name: "Chuột không dây", price: 25.0), // Giá rẻ
  Product(id: "p3", name: "Bàn phím cơ", price: 80.0),
  Product(id: "p4", name: "Lót chuột", price: 10.0),       // Giá rẻ
  Product(id: "p5", name: "Màn hình 4K", price: 450.0),
];
```

BƯỚC 4: Hiển thị danh sách (Collection .map Logic)

👉 Tạo file mới: `lib/services/product_service.dart`

```
// lib/services/product_service.dart
import '../models/product.dart';
import '../data/data_source.dart';

class ProductService {
  /// Lấy toàn bộ sản phẩm
  List<Product> getAllProducts() {
    return rawProducts;
  }

  /// Lấy sản phẩm giá rẻ (< 100)
  List<Product> getCheapProducts() {
    return rawProducts.where((p) => p.price < 100).toList();
  }

  /// Xử lý khi người dùng chọn sản phẩm
  void handleAddToCart(Product product) {
    print("Đã chọn mua: ${product.name}");
    print("Giá tiền: ${product.price}");
  }
}
```

💡 GIẢI THÍCH

- Đây là **Dart thuần**
- Không biết UI là gì
- Sau này:
  - Có thể test
  - Có thể đổi UI mà không sửa logic

BƯỚC 5: KẾT NỐI UI VỚI LOGIC THÔNG QUA FUNCTION & CALLBACK

- **Mục tiêu:** cách UI giao tiếp với logic thông qua Function & Callback.

📌 `lib/main.dart`

Dart

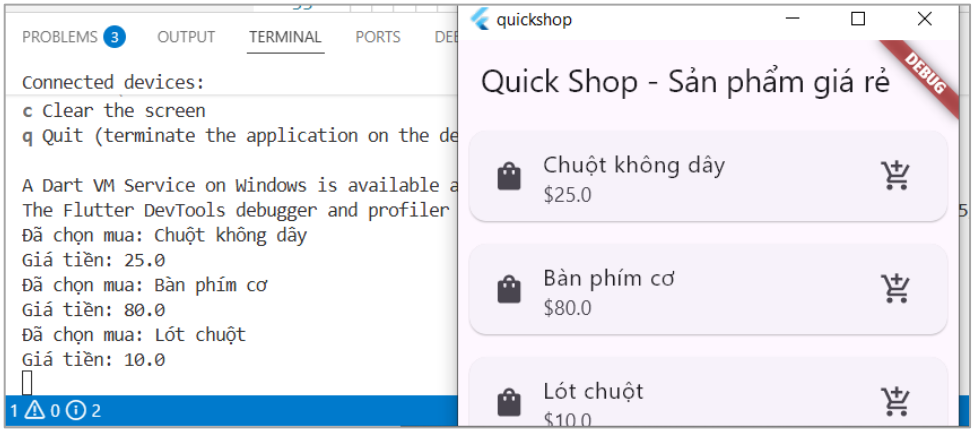
```
import 'package:flutter/material.dart';
import 'models/product.dart';
import 'services/product_service.dart';

void main() {
  runApp(MaterialApp(home: QuickShopPage()));
}

class QuickShopPage extends StatelessWidget {
```

```
  QuickShopPage({super.key});

  // 1 Tạo service (logic)
  final ProductService _service = ProductService();

  @override
  Widget build(BuildContext context) {
    // ❌ KHÔNG xử lý logic ở đây
    // ✅ Chỉ lấy KẾT QUẢ đã xử lý
    final List<Product> products = _service.getCheapProducts();

    return Scaffold(
      appBar: AppBar(title: const Text("Quick Shop - Sản phẩm giá rẻ")),
      body: ListView(
        children: products.map((product) {
          return Card(
            margin: const EdgeInsets.all(8),
            child: ListTile(
              leading: const Icon(Icons.shopping_bag),
              title: Text(product.name),
              subtitle: Text("\$${product.price}"),
              trailing: const Icon(Icons.add_shopping_cart),

              // UI chỉ GỌI logic
              onTap: () => _service.handleAddToCart(product),
            ),
          );
        }).toList(),
      ),
    );
  }
}
```

```
vices > product_service.dart > ...
class ProductService {
  /// Lấy toàn bộ sản phẩm
  List<Product> getAllProducts() {
    return rawProducts;
  }

  /// Lấy sản phẩm giá rẻ (< 100)
  List<Product> getCheapProducts() {
    return rawProducts.where(test: (Product p) => p.price < 100).toList();
  }

  /// Xử lý khi người dùng chọn sản phẩm
```

```
@override
Widget build(BuildContext context) {
  // ❌ KHÔNG xử lý logic ở đây
  // ✅ Chỉ lấy KẾT QUẢ đã xử lý
  final List<Product> products = _service.getCheapProducts();

  return Scaffold(
    appBar: AppBar(title: const Text(data: "Quick Shop - Sản phẩm giá rẻ")),
    body: ListView(
      children: products.map<Card>(toElement: (Product product) {
        return Card(
          margin: const EdgeInsets.all(value: 8)
```


PhucPT10@fpt.edu.vn



⚠ LƯU Ý

✕ TUYỆT ĐỐI KHÔNG LÀM

```
build() {
  rawProducts.where(...).map(...);
}
```

☑ PHẢI LÀM

```
// Logic xử lý
service.getCheapProducts();

// UI hiển thị
ListView(...)
```

BƯỚC 6: Nâng cao - Lọc dữ liệu (Collection .where Logic)

- **Mục tiêu:** Chỉ hiện sản phẩm giá rẻ (< 100$).
- **Thao tác:** Sửa dòng khai báo displayList trong main.dart.

Dart

📌 BƯỚC 6.1 – THÊM LOGIC LỌC VÀO SERVICE

📍 **File:** `lib/services/product_service.dart`

```
class ProductService {
  // ... code cũ giữ nguyên

  /// Lọc sản phẩm theo giá tối đa
  List<Product> filterByMaxPrice(double maxPrice) {
    return rawProducts.where((product) {
      return product.price < maxPrice;
    }).toList();
  }
}
```

💡 GIẢI THÍCH CHO SINH VIÊN

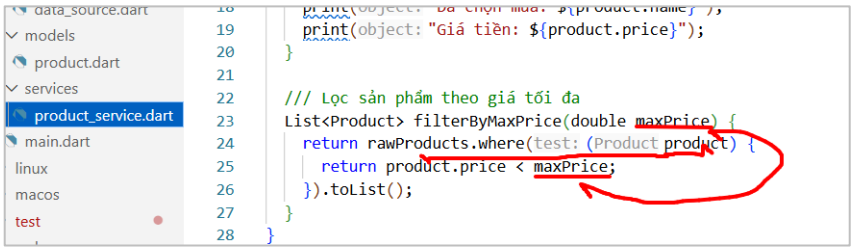
- `.where()` là **logic xử lý dữ liệu**
- Logic → **phải ở service**
- UI **không cần biết** lọc kiểu gì

PhucPT10@fpt.edu.vn

BƯỚC 6.2 – UI GỌI LOGIC, KHÔNG XỬ LÝ

File: `lib/main.dart`

```
class QuickShopPage extends StatelessWidget {
    QuickShopPage({super.key});

  final ProductService _service = ProductService();

  @override
  Widget build(BuildContext context) {
    // ✅ UI chỉ nhận KẾT QUẢ ĐÃ LỌC
    final products = _service.filterByMaxPrice(100);

    return Scaffold(
      appBar: AppBar(
        title: const Text("Sản phẩm giá rẻ (< 100\$)"),
      ),
      body: ListView(
        children: products.map((product) {
          return Card(
            margin: const EdgeInsets.all(8),
            child: ListTile(
              title: Text(product.name),
              subtitle: Text("\$${product.price}"),
              onTap: () => _service.handleAddToCart(product),
            ),
          );
        }).toList(),
      ),
    );
  }
}
```



PhucPT10@fpt.edu.vn

🧠 3. GIẢI THÍCH LUỒNG HOẠT ĐỘNG (RẤT QUAN TRỌNG)

🔄 Trình tự thực thi

```
UI build()
↓
Gọi service.filterByMaxPrice(100)
↓
Service xử lý .where()
↓
Trả về List<Product>
↓
UI chỉ hiển thị
```

👉 **UI KHÔNG xử lý logic**

7 KIỂM TRA & ĐÁNH GIÁ KẾT QUẢ

1. **Chạy ứng dụng:** Nhấn F5 (Run).
2. **Kiểm tra UI:**
   - Màn hình có hiện danh sách không?
   - Nếu làm Bước 6, danh sách có chỉ hiện "Chuột", "Bàn phím", "Lót chuột" không? (Mất Laptop và Màn hình là đúng).
3. **Kiểm tra Logic:**
   - Bấm vào một dòng sản phẩm.
   - Nhìn xuống **Debug Console**. Có thấy dòng chữ: I/flutter (xxxx): Đã chọn mua: Chuột không dây không?
   - Nếu thấy -> Bạn đã thành công kết nối UI và Logic!

8 LỖI PHỔ BIẾN & CÁCH SỬA (DEBUGGING)

Lỗi (Error)	Dấu hiệu	Cách sửa
Quên .toList()	type 'MappedListIterable' is not a subtype of type 'List<Widget>'	Thêm .toList() vào cuối hàm .map(...).
Lỗi Null Safety	Null check operator used on a null value	Kiểm tra Constructor trong Class Product, đảm bảo có required và dữ liệu không bị null.
Lỗi Callback	Hàm chạy ngay khi mở app dù chưa bấm nút.	Sai: onTap: hamXuLy(). Đúng: onTap: hamXuLy (bỏ ngoặc tròn) hoặc onTap: () => hamXuLy().

9 KẾT QUẢ ĐẠT ĐƯỢC

Sau bài này, bạn đã thực sự hiểu:

- ✅ **Class Product** không phải code chơi, nó là khung xương để hiển thị ListTile.
- ✅ **List rawProducts** là nguồn dữ liệu.
- ✅ **.map()** là cái máy chuyển đổi từ Dữ liệu sang Giao diện.
- ✅ **onTap function** là nơi chứa logic nghiệp vụ.

10 BÀI TẬP MỞ RỘNG (CHALLENGES)

Dành cho bạn nào muốn "lên trình":

PhucPT10@fpt.edu.vn

1. **Level 1 (Logic Hàm):** Viết một hàm riêng void handleAddToCart(Product p) bên ngoài hàm build, rồi gọi nó trong onTap.
2. **Level 2 (Logic Toán):** Tính tổng tiền của các sản phẩm đang hiển thị và in ra Console (Gợi ý: Dùng vòng for-in duyệt qua cheapProducts).
3. **Level 3 (Logic Class):** Thêm thuộc tính String imageUrl vào class Product. Tìm link ảnh trên mạng điền vào data, và hiển thị ảnh đó bằng Image.network(product.imageUrl) trong ListTile.

(Sơ đồ minh họa quá trình: Data Object -> .map() -> Widget)

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU CHUYÊN SÂU

10 PROMPT PHÂN TÍCH – ĐÁNH GIÁ (LEVEL KHÁ – GIỎI)

Sinh viên đã:

- Học xong **kiến thức đầy đủ Slot 3** (Prompt 4)
- Được cảnh báo lỗi & best practices (Prompt 5)

Giờ đây, sinh viên cần:

- Tự nghiên cứu **sâu hơn mức bắt buộc**
- Phát triển tư duy:
  - Phân tích
  - So sánh
  - Đánh giá
  - Lựa chọn giải pháp

👉 Các prompt dưới đây được thiết kế để sinh viên:

- **Tự đặt câu hỏi đúng**
- **Tự tranh luận với AI**
- **Chuẩn bị nền tảng cho project Flutter lớn hơn**

YÊU CẦU CHUNG CHO TẤT CẢ PROMPT

Khi trả lời mỗi prompt, sinh viên **bắt buộc** phải:

- Giải thích **vì sao chọn giải pháp**
- Chỉ ra **ưu / nhược điểm**
- Nêu **khi nào nên / không nên áp dụng**
- Liên hệ đến **ứng dụng Flutter thực tế**

Prompt 1 – Thiết kế Function có trách nhiệm rõ ràng

Phân tích sự khác biệt giữa:

- Một function "làm mọi thứ"
- Nhiều function nhỏ, mỗi function một nhiệm vụ

Trong một app Flutter thực tế:

- Khi nào nên tách function?
- Khi nào tách quá mức lại gây khó đọc?

Đánh giá ảnh hưởng đến:

- Khả năng maintain
- Khả năng test
- Khả năng mở rộng

Prompt 2 – Callback & Luồng xử lý logic

Phân tích vai trò của **callback function** trong Flutter:

- Callback giúp tách UI và logic như thế nào?
- Nếu lạm dụng callback thì vấn đề gì xảy ra?

So sánh:

- Viết logic trực tiếp
- Truyền callback

Đánh giá trong bối cảnh app có nhiều màn hình.

Prompt 3 – Thiết kế Class & Trách nhiệm

Phân tích một class Dart dùng làm model:

- Nên chứa những gì?
- Không nên chứa những gì?

Đánh giá hậu quả khi:

- Class chứa quá nhiều logic
- Class chỉ là "data holder" thuần túy

Liên hệ đến khả năng mở rộng app Flutter.

◈ Prompt 4 – Constructor & Tính rõ ràng của object

So sánh:

- Constructor thường
- Named constructor

Trong dự án Flutter thật:

- Khi nào named constructor giúp code dễ hiểu hơn?
- Khi nào lại gây dư thừa?

Đánh giá ảnh hưởng đến readability & bug logic.

◈ Prompt 5 – Collections & Chuẩn bị dữ liệu cho UI

Phân tích vai trò của:

- List
- Map

Trong việc chuẩn bị dữ liệu cho UI Flutter:

- Khi nào nên dùng map()?
- Khi nào nên dùng vòng lặp for truyền thống?

Đánh giá:

- Độ rõ ràng
- Hiệu suất
- Khả năng debug

◈ Prompt 6 – map(), where() và tính dễ đọc của code

So sánh 2 cách xử lý collection:

- Dùng map(), where() liên tiếp
- Dùng vòng lặp tường minh

Phân tích:

- Khi nào functional style giúp code đẹp hơn?
- Khi nào lại khiến code khó hiểu với team?

◈ Prompt 7 – Lỗi logic không báo lỗi trong Dart

Phân tích các lỗi Dart:

- Code chạy
- Không báo lỗi
- Nhưng kết quả sai

Vì sao các lỗi này:

- Rất nguy hiểm trong Flutter app?

Đề xuất chiến lược:

- Viết code phòng ngừa
- Kiểm tra logic sớm

◈ Prompt 8 – Đặt tên & giao tiếp trong code

Phân tích tầm quan trọng của:

- Tên function
- Tên class
- Tên biến

Đánh giá:

- Một cái tên kém có thể gây lỗi tư duy như thế nào?
- Ảnh hưởng đến onboarding thành viên mới?

◈ Prompt 9 – Chuẩn bị Dart core cho project lớn

Giả sử bạn bắt đầu một project Flutter sẽ:

- Có 20+ màn hình
- Nhiều luồng xử lý dữ liệu

Phân tích:

- Những quyết định Dart core nào ở giai đoạn đầu ảnh hưởng lâu dài?
- Những sai lầm nào rất khó sửa về sau?

◈ Prompt 10 – Tư duy Dart Core của lập trình viên giỏi

Theo bạn, sự khác biệt lớn nhất giữa:

- Sinh viên biết cú pháp Dart
- Lập trình viên Flutter giỏi Dart core

Phân tích:

- Tư duy
- Cách tổ chức code
- Cách phòng ngừa lỗi

Liên hệ trực tiếp với khả năng làm project thật.

🎯 VAI TRÒ CỦA PROMPT 6 TRONG CHUỖI BÀI GIẢNG

Prompt này giúp sinh viên:

- Chuyển từ:
  "Học để làm bài tập"
  → "Học để thiết kế giải pháp"
- Phát triển tư duy:
  - Phân tích
  - Đánh giá
  - Lựa chọn
- Chuẩn bị nền tảng cho:
  - Slot tiếp theo (UI, State, Architecture)
  - Project Flutter quy mô lớn

PhucPT10@fpt.edu.vn

1 10 PROMPT THỰC TIỄN CHUYÊN NGHIỆP (PROJECT-SCALE THINKING)

Mỗi prompt buộc sinh viên phải:

- Phân tích
- Đánh giá rủi ro
- Đề xuất hướng giải quyết
- Liên hệ môi trường làm việc thật

◈ Prompt 1 – Function phình to theo thời gian

Trong một dự án Flutter lớn:

- Một function ban đầu rất nhỏ nhưng dần phình to theo yêu cầu

Phân tích:

- Dấu hiệu function đã "quá tải trách nhiệm"
- Khi nào nên refactor?
- Nếu không refactor sớm thì hậu quả dài hạn là gì?

◈ Prompt 2 – Callback hell ở cấp độ Dart Core

Phân tích hiện tượng:

- Callback lồng nhau nhiều cấp

Trong dự án lớn:

- Vì sao callback dễ bị lạm dụng?
- Ảnh hưởng đến:
  - Readability
  - Debug
  - Review code

Đề xuất cách tổ chức lại logic.

◈ Prompt 3 – Class "God Object" trong Dart

Phân tích một class Dart:

- Chứa dữ liệu
- Chứa logic xử lý
- Chứa cả rule nghiệp vụ

Vì sao "God class" rất nguy hiểm trong dự án lớn?

- Dấu hiệu sớm
- Chi phí sửa chữa về sau

◈ Prompt 4 – Constructor phức tạp & bug tiềm ẩn

Một class có:

- Quá nhiều tham số constructor

Phân tích:

- Vì sao dễ sinh bug logic?
- Named constructor giúp được gì?
- Khi nào named constructor không còn đủ?

◈ Prompt 5 – Collections lớn & logic khó kiểm soát

Trong dự án lớn, dữ liệu thường:

- Lớn
- Nhiều tầng

Phân tích:

- Khi nào dùng map(), where() khiến code khó đọc?
- Khi nào vòng lặp tường minh lại tốt hơn?

PhucPT10@fpt.edu.vn

Đánh giá góc nhìn team.

◈ Prompt 6 – Code "đẹp" nhưng khó debug

Functional style trong Dart giúp code ngắn gọn.
Phân tích:

- Vì sao code càng "ngắn" càng khó debug?
- Khi nào nên hy sinh sự gọn gàng để đổi lấy rõ ràng?

◈ Prompt 7 – Dart Core & Code Review

Trong code review dự án Flutter:

- Reviewer thường nhìn gì đầu tiên trong function / class?

Phân tích:

- Dấu hiệu của code Dart "non-professional"
- Dấu hiệu của code Dart "production-ready"

◈ Prompt 8 – Quyết định Dart Core ảnh hưởng lâu dài

Phân tích:

- Những quyết định Dart Core nào ở giai đoạn đầu project:
    - Rất khó sửa về sau?

Ví dụ:

- Cách tổ chức function
- Cách thiết kế class
- Cách xử lý collection

◈ Prompt 9 – Dart Core trong teamwork

Trong team nhiều developer:

- Mỗi người một phong cách Dart

Phân tích:

- Vì sao inconsistency rất nguy hiểm?
- Vai trò của:
    - Naming
    - Code convention
    - Review

◈ Prompt 10 – Tư duy Dart Core của Senior

So sánh:

- Junior viết Dart Core
- Senior viết Dart Core

Phân tích sự khác biệt về:

- Tư duy
- Cách phòng ngừa lỗi
- Cách chuẩn bị cho mở rộng hệ thống

🎓 VAI TRÒ CỦA PROMPT 7 TRONG CHUỖI BÀI GIẢNG

Prompt này giúp sinh viên:

- Nhận thức rõ:
  "Dart Core không phải kiến thức phụ – nó quyết định chất lượng toàn bộ app"
- Bắt đầu suy nghĩ:
    - Như một **developer chuyên nghiệp**
    - Như một **thành viên trong team lớn**
- Chuẩn bị nền tảng tư duy cho:
    - Architecture

PhucPT10@fpt.edu.vn

- State management
- Project Flutter quy mô công nghiệp

PhucPT10@fpt.edu.vn